Chapter 4

SQL-Fundamentals

4.1 INTRODUCTION

"Structured Query Language" is abbreviated as SQL. SQL was designed to be "human-friendly".

Most of its commands resemble spoken English. This feature of SQL makes it easy to read, understand, and learn. The SQL language may be considered one of the major reasons for the success of relational databases in the commercial world. The Structured Query Language is a relational database language. DBMS is not made by SQL. SQL is a medium used to communicate with the DBMS.

SQL is referred to as nonprocedural database language. To retrieve data from the database using nonprocedural language it is enough to tell what data to be retrieved, rather than how to retrieve it. The DBMS will take care of locating the information in the database. Commercial database management systems allow SQL to be used in two distinct ways.

1. SQL commands can be typed at the command line directly. The DBMS interprets and processes the SQL commands immediately, and the results are displayed. This method of SQL processing is called interactive SQL.
2. The second method is called programmatic SQL. Here, SQL statements are embedded in a host language such as COBOL, FORTRAN, C, etc. SQL needs a host language because SQL is not a really complete computer programming language as such because it has no statements or constructs that allow branch or loop. The host language provides the necessary looping and branching structures and the interface with the user, while SQL provides the statements to communicate with the DBMS.

4.2 ADVANTAGES OF SQL

- SQL is a high level language that provides a greater degree of abstraction than procedural languages.
- SQL enables the end-users and systems personnel to deal with a number of database management systems.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

- SQL specifies what is required and not how it should be done.
- SQL is simple and easy to learn.
- SQL handles complex situations easily.
- All SQL operations are performed at a set level.
- Reduced training cost
- Enhanced productivity
- Application portability: Application portability means applications can be moved from machine to machine when each machine uses SQL.
- Application durability: A standard language tends to remain for a long time, hence there will be little pressure to rewrite old applications.
- Reduced dependence on a single vendor

4.3 PARTS OF SQL

The SQL language has several parts:

- **Data-definition language** (DDL). The SQL DDL provides commands for defining relation schemas, deleting relations, and modifying relation schemas.
- **Interactive data-manipulation language** (DML). The SQL DML includes a query language based on both the relational algebra and the tuple relational calculus. It also includes commands to insert tuples into, delete tuples from, and modify tuples in the database.
- **View definition**. The SQL DDL includes commands for defining views.
- **Transaction control**. SQL includes commands for specifying the beginning and ending of transactions.
- **Embedded SQL** and **dynamic SQL**. Embedded and dynamic SQL define how SQL statements can be embedded within general-purpose programming languages, such as C, C++, Java, PL/I, COBOL, Pascal, and FORTRAN.
- **Integrity**. The SQL DDL includes commands for specifying integrity constraints that the data stored in the database must satisfy. Updates that violate integrity constraints are disallowed.
- **Authorization**. The SQL DDL includes commands for specifying access rights to relations and views.

4.4 DOMAIN TYPES IN SQL

In relational model the data are stored in the form of tables. A table is composed of rows and columns. When we create a table we must specify a data type for each of its columns. These data types define the domain of values that each column can take. A number of built-in data types are available. Some of the built-in data types are string data type to store characters, number data type to store numerical value, and date and time data type to store when the event happened.

1. **Char (n):** The CHAR data type specifies a fixed-length character string. The syntax of CHAR data type is:

   CHAR (n) – Fixed length character data, "n" characters long.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

"n" specifies the character length. If a value shorter than the column length is inserted then the input is padded with space to the column length. If a value greater than column length is inserted then an error is returned.

2. **varchar(n):** Variable length character strings, with user-specified maximum length *n.* The syntax of varchar data type is:

   VARCHAR2 (n) – Variable length character of "n" length.

   "n" specifies the character length.
3. **int:** Integer (a finite subset of the integers that is machine-dependent).
4. **Smallint:** Small integer (a machine-dependent subset of the integer domain type).
5. **Numeric (p,d):** fixed point number, where "p" is the total number of digits and "q" is the number of digits to the right of decimal point.
6. **Real, double precision:** Floating point and double-precision floating point numbers, with machine-dependent precision.
7. **float (n):** Floating point number, with user-specified precision of at least *n* digits.
8. **Date:** Dates, containing a (4 digit) year, month and date
   Example: date '2005-7-27'
9. **Time:** Time of day, in hours, minutes and seconds.
   Example: time '09:00:30' time '09:00:30.75'
10. **Timestamp:** date plus time of day
    Example: timestamp '2005-7-27 09:00:30.75'
11. **Interval:** period of time
    Example: interval '1' day

4.5 TERMINOLOGY

A table is the basic storage unit of DBMS. The table stores the necessary data. The common terminologies used in relational databases are as follows:

1. A **row or tuple** represents all the data required for a particular entity. Each row in a table is identified using primary key.
2. A **column or attribute** represents each column in the table. Each column represents a property or attribute of an entity. The columns can a key attribute or non key attribute.
3. A **field** can be intersection of column and a row. There can be only one value in it

4.6 DATA DEFINITION LANGUAGE (DDL)

A DDL is a language used to define data structures within a database. It is considered to be the subset of Structured Query Language. A Data Definition Language has a pre-defined syntax for describing data.

For example, to create a new table using SQL syntax, the CREATE command is used, followed by parameters for the table name and column definitions. The DDL can also define the name of each column and the associated data type.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Once a table is created, it can be modified using the ALTER command. If the table is no longer needed, the DROP command will delete the table.

Since DDL is a subset of SQL, it does not include all the possible SQL commands. For example, commands such as SELECT and INSERT are considered part of the Data Manipulation Language (DML), while access commands such as CONNECT and EXECUTE are part of the Data Control Language (DML).

The various **DDL commands** are as follows:

- CREATE: to create objects in the database
- DESC: describes the structure of the database
- ALTER: alters the structure of the database
- DROP: removes a table from the database
- TRUNCATE: removes all records from a table, including all spaces allocated for the records are removed
- RENAME: renames an object

The different structures that are created by DDL are Tables, Views, Sequences, Triggers, Indexes, etc.

4.6.1 DDL Commands

1. **CREATE:** Command is used for creating tables.

   Syntax:

   create table<*table name*> *(columnname 1 data type (size), Columnname 2 data type(size),.., columnname n data type(size));*

   Example: create table customer (cust_name varchar2(15), social_security_no number(11), cust_street varchar2(7), cust_city varchar2(10));

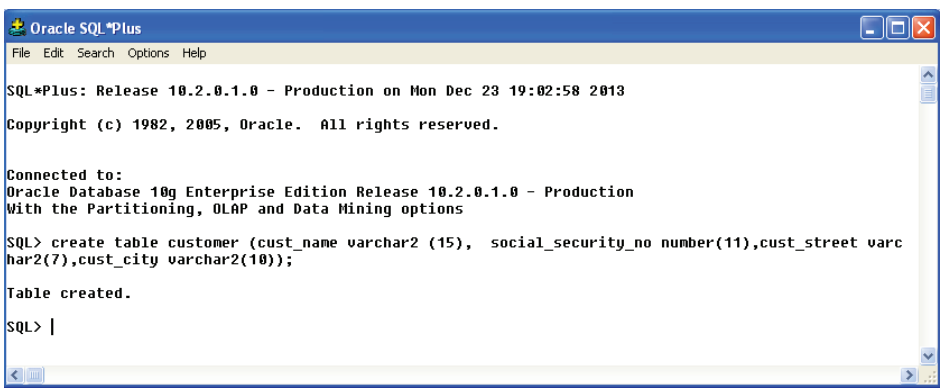


Fig. 4.1. *Table creation.*

2. **DESC:** Command is used to view the table structure.

   Syntax:

   desc<*table name*>*;*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Example: desc customer;

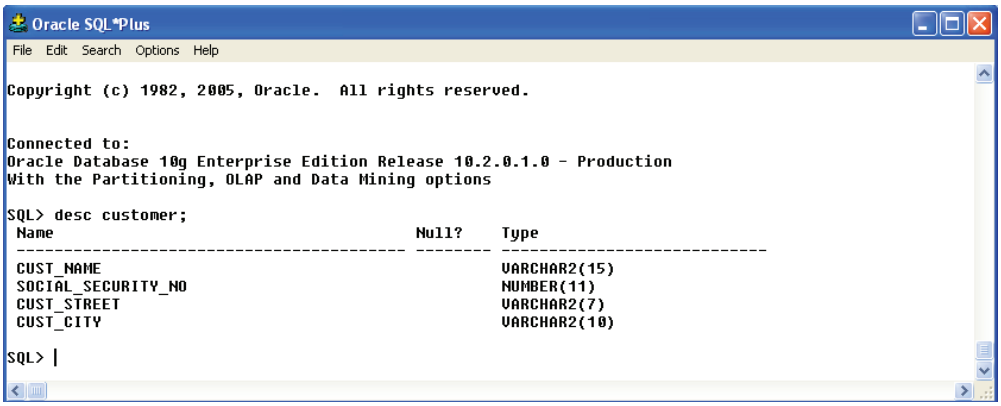


Fig. 4.2. *Table description.*

3. **ALTER:** Command is used for modifying table structure.

(*i*) Syntax:

alter table<*table name*>***modify***(*columnname data type (new size)*);

Example: alter table customer modify (cust_street varchar2 (10));

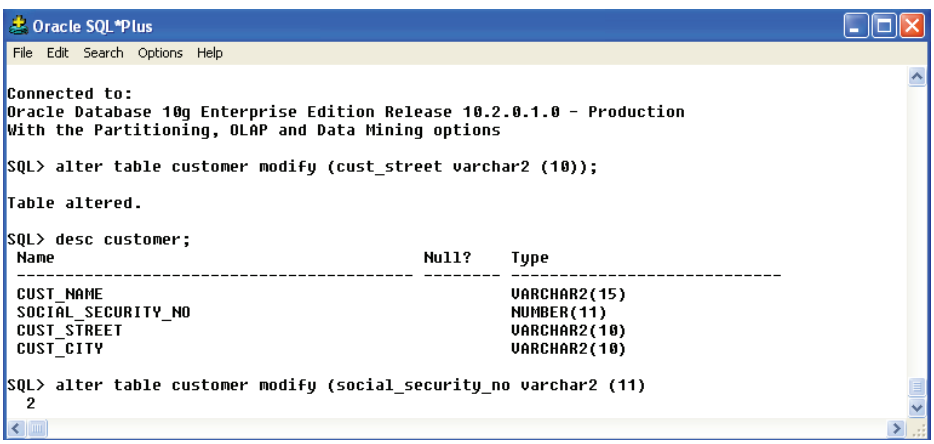


Fig. 4.3. *Modification of size data type.*

(*ii*) Syntax:

alter table <*table name*> ***modify*** (*columnname new data type (size)*);

Example: alter table customer modify (social_security_no varchar2 (11)**);**

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

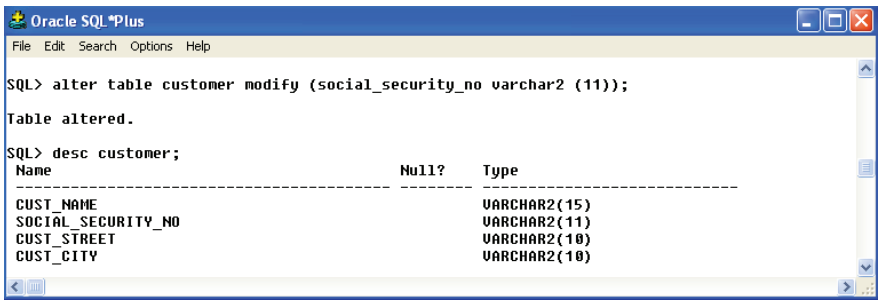


Fig. 4.4. *Modification of data type.*

(*iii*) Syntax:

alter table<*table name*>***add***(*new columnname data type (size)*);

Example: alter table customer add (acc_no varchar2(5));

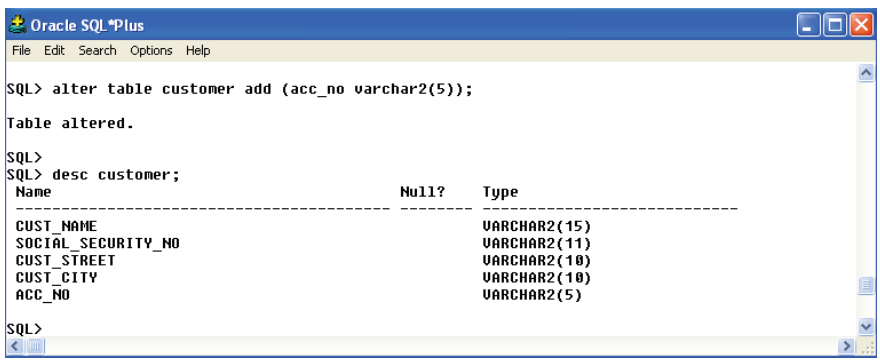


Fig. 4.5. *Modification of column name.*

(*iv*) Syntax:

alter table<*table name*> ***drop*** *(column name);*

Example: alter table customer drop (acc_no);

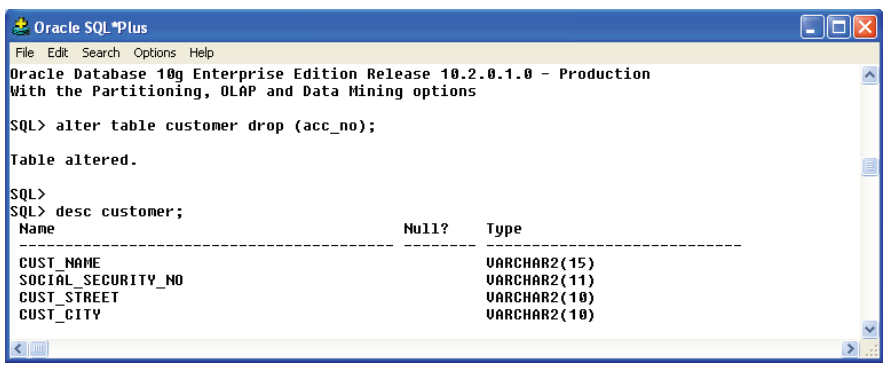


Fig. 4.6. *Modifying the table by dropping a column.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4. **RENAME:** This command is used to change the name of the table.

   Syntax:

   rename<*Old table name*>***to***<*New table name*>*;*

   Example: rename customer to customer1;

5. **DROP:** The DROP command removes a table from the database. All the tables' rows, indexes and privileges will also be removed. No DML triggers will be fired. The operation cannot be rolled back.

   Syntax:

   drop table<*table name*>*;*

   Example: drop table customer1;

6. **TRUNCATE:** The **TRUNCATE** command removes **all rows** from a table. The operation cannot be rolled back and no triggers will be fired. TRUNCATE is faster.

   Syntax:

   truncate table<*table name*>*;*

   Example: **truncate** table customer1;

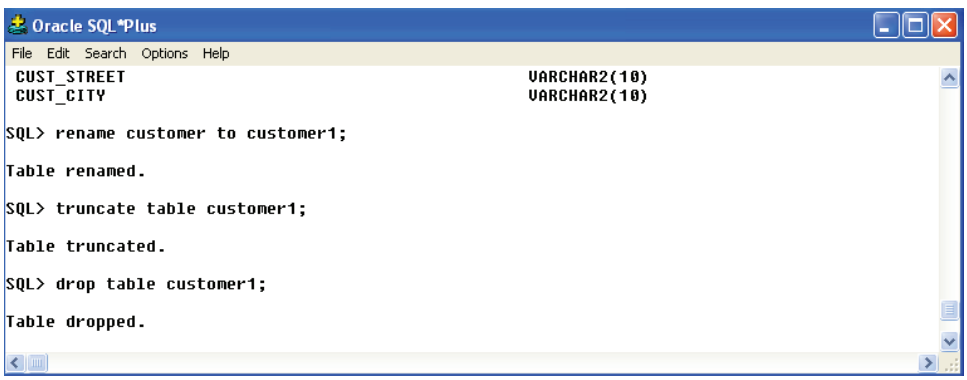


Fig. 4.7. *Figure depicting rename, truncate and drop operations.*

4.7 DATA MANIPULATION LANGUAGE (DML)

Data Manipulation Language (DML) statements are used for managing data within schema objects. Data Manipulation language commands let user to insert, modify and delete the data from database.

DML Commands

Consider the following table for understanding the DML commands

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

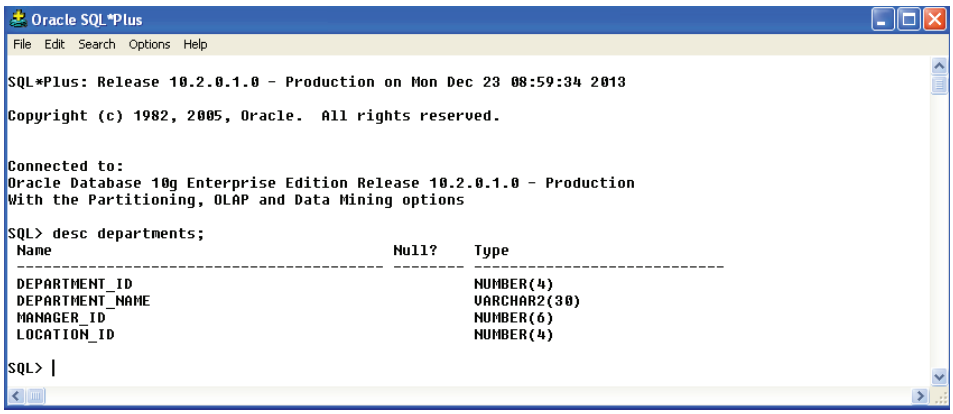


Fig. 4.8. *Table departments.*

1. **INSERT:** To insert one or more number of Rows

Syntax 1:

insert into <table name> values *(List of Data Values);*

Example: insert into departments values(‘10’,‘administration’,‘300’,‘1300’);

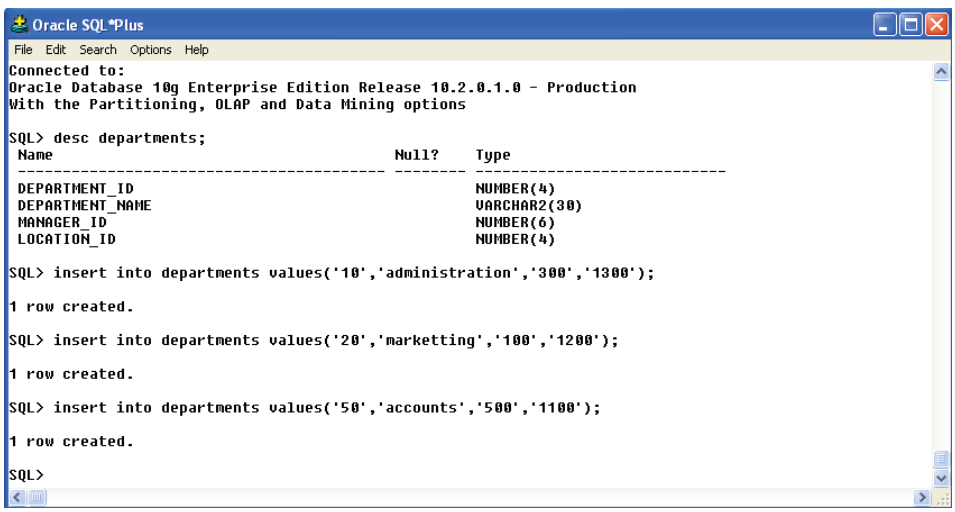


Fig. 4.9. *Inserting values into all columns of departments table.*

Syntax 2:

insert *into <table name> (column names)* ***values*** *(list of data values);*

Examples:

1. insert into departments (department_id,department_name,manager_id,location_id) values(‘60’,‘IT’,‘101’,‘1300’);
2. insert into departments (department_id,department_name,location_id) values(‘30’,‘sales’,‘1111’);

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Using this insert statement value of available columns can alone be inserted. In our first example we have inserted values for all the columns in the departments table but in the second example value for manager_id are not inserted.

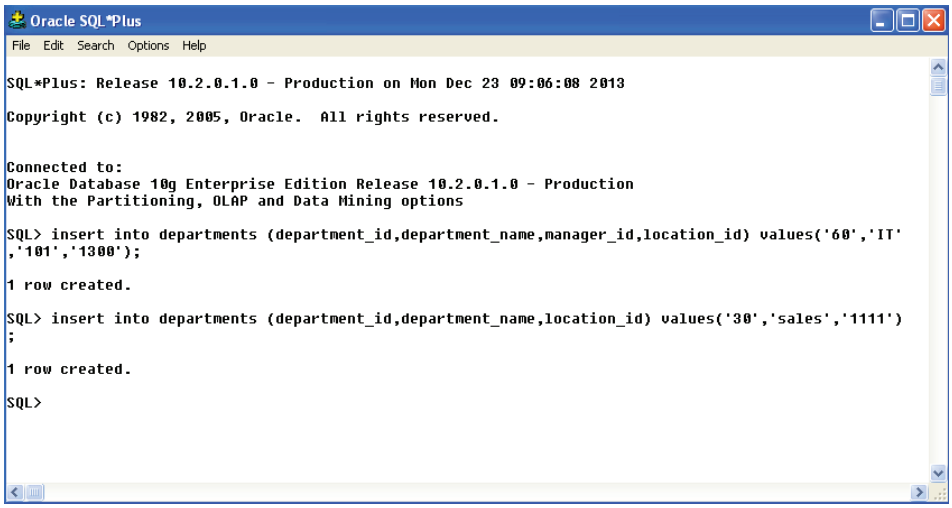


Fig. 4.10. *Inserting values into specified columns of the departments table.*

Insert command using User interaction

Syntax 3:

insert *into <Table name>* ***values*** *(&columnname1, &columnname2…);*

Example: insert into departments values (&department_id,&department_name,&manager_id,&location_id);

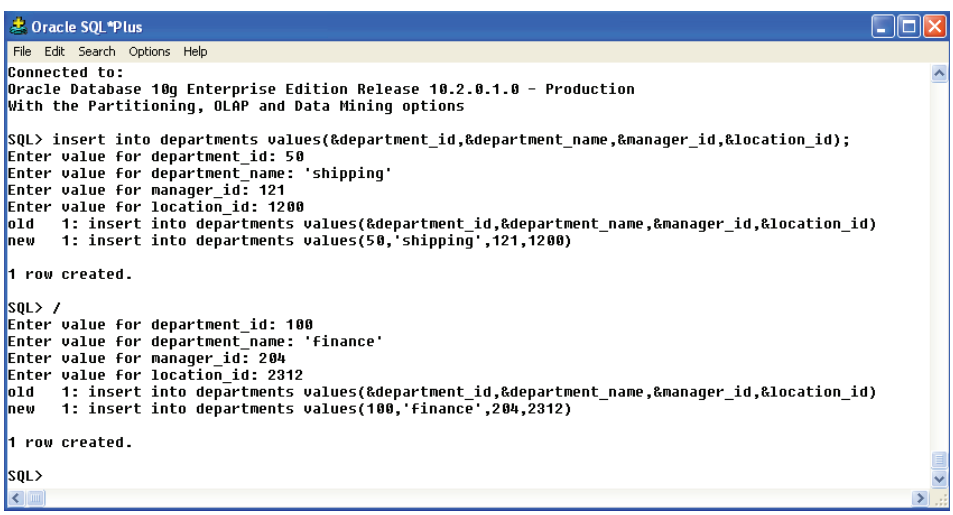


Fig. 4.11. *Inserting values into departments table using user interaction.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

While inserting values into the table, values inserted should be enclosed in single quotes. For numerical values it's optional.

2. **SELECT**: To display one or more rows

Syntax 1:

select * from <*table name*>;

Example: select * from departments;

This query selects all the columns present in the specified table.

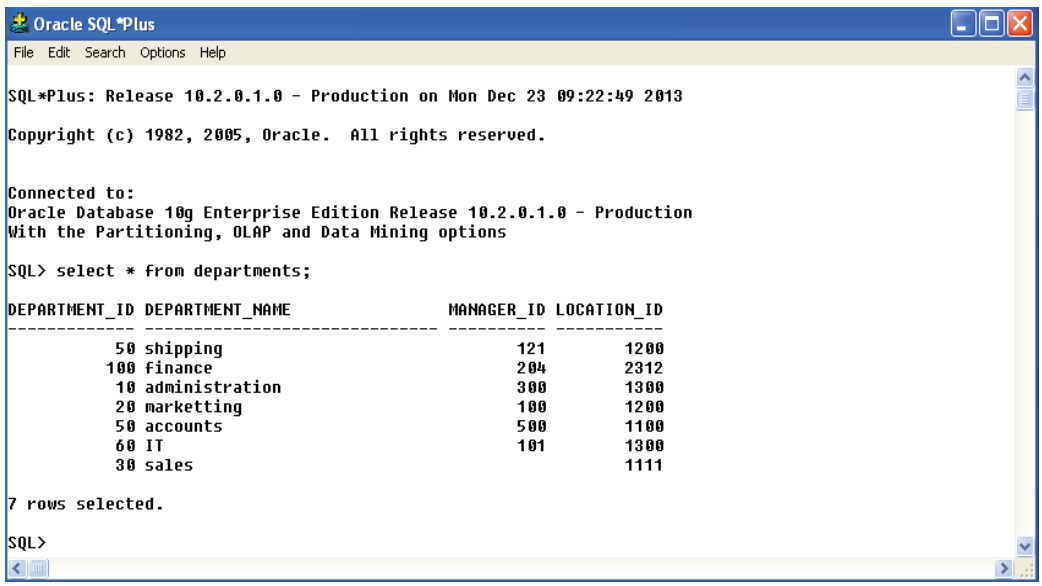


Fig. 4.12. *Selecting and displaying all values present in a table.*

Syntax 2:

select columnname1, columnname 2 **from** <table name>;

Example: select department_id, department_name from departments;

This select query helps us to select and display the required columns from the table. If the original table contains 10 columns and if the user wants to view only two or three then this type of select statement can be used. In our example columns department_id and department_name is selected from the departments table.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

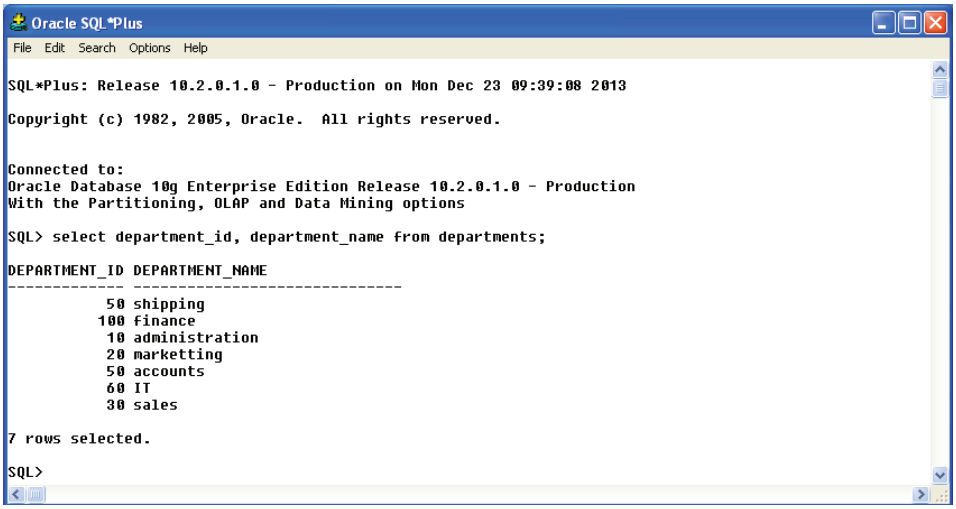


Fig. 4.13. *Displaying selected columns from a table.*

Syntax 3:

select * from <*table name*> ***where*** <*condition*>*;*

Example: select * from departments where department_name='IT';

This type of select statements helps the user to select values from the table based on some condition given in where clause. In our example we have selected the details of department_ name='IT';

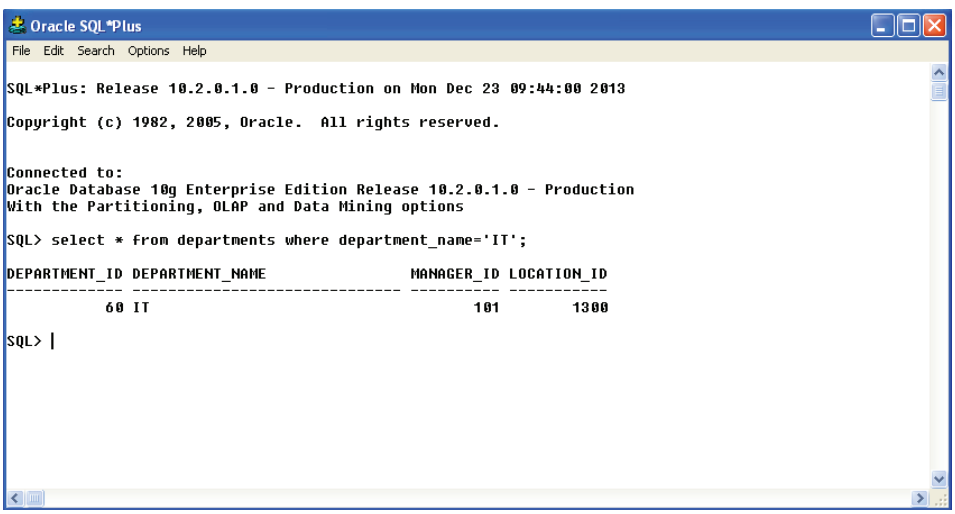


Fig. 4.14. *Selecting and displaying column values using some condition.*

3. **UPDATE:** Used to alter the column values in a table. In the Update statement, WHERE clause identifies the rows that get affected. If you do not include the WHERE clause, column values for all the rows get affected.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Syntax:

update *<table name>* ***set*** *column_name=new_value where <condition>;*

Example: **update** departments set manager_id=120 where department_id=60;

In our example the manager_id is affected for the department with department_id=60.

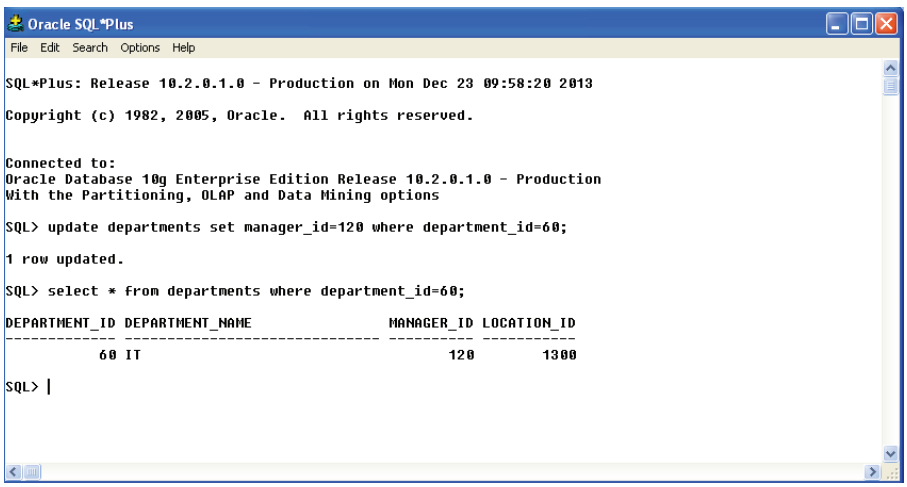


Fig. 4.15. *Updating column values based on a condition.*

4. **DELETE:** Used to delete one or more rows.

Syntax:

delete *from <table name>* ***where*** *<condition>;*

Example: **delete** from departments **where** department_name='IT';

In out example the entire details of department with department_name='IT' will be deleted from the table departments.

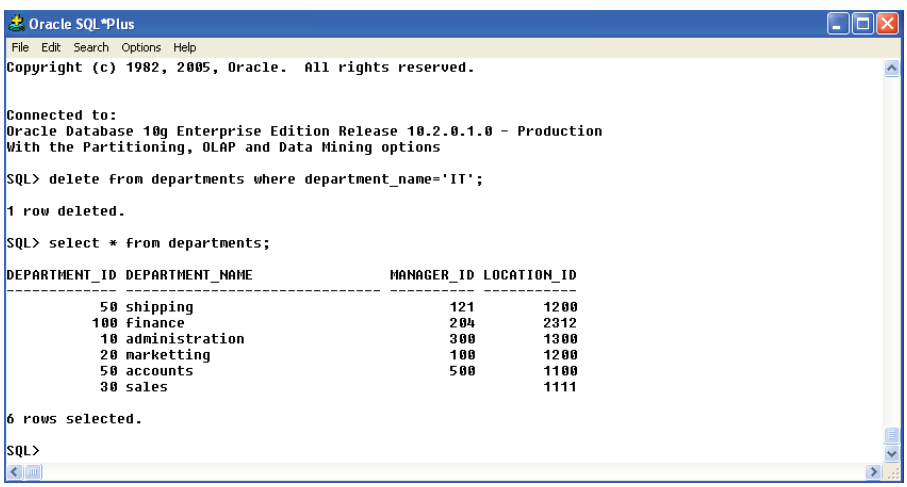


Fig. 4.16. *Deleting a value from a table based on a condition.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.8 BASIC STRUCTURE OF SQL EXPRESSION

SQL expression consists of three clauses:

- **Select:** The select clause corresponds to projection operation of the relational algebra. The SELECT statement is used to query the database and retrieve the fields that you specify. You can select as many fields (column names) as you want, or use the asterisk symbol "*" to select all fields.
- **From:** The from clause corresponds to the Cartesian product operation of the relational algebra. It specifies the table names that will be queried to retrieve the desired data.
- **Where:** The where clause corresponds to the selection predicate of the relations that appear in the from clause. The WHERE clause (optional) specifies which data values or rows will be returned or displayed, based on the criteria you specify.

4.8.1 General Form of SQL Query

Select *A1, A2……., An*
From *R1, R2……., Rm*
Where *P*

Where, A1 = represent attributes
R1 = represent relation
P = is a predicate

4.8.2 Selecting all Columns from a Relation

Syntax:

*Select * from <table name>;*

Example: Select * from departments;

Oracle SQL*Plus

File Edit Search Options Help

```
SQL> Select * from departments;

DEPARTMENT_ID DEPARTMENT_NAME                MANAGER_ID LOCATION_ID
------------- ------------------------------ ---------- -----------
           10 Administration                        200        1700
           20 Marketing                             201        1800
           30 Purchasing                            114        1700
           40 Human Resources                       203        2400
           50 Shipping                              121        1500
           60 IT                                    103        1400
           70 Public Relations                      204        2700
           80 Sales                                 145        2500
           90 Executive                             100        1700
          100 Finance                               108        1700
          110 Accounting                            205        1700

DEPARTMENT_ID DEPARTMENT_NAME                MANAGER_ID LOCATION_ID
------------- ------------------------------ ---------- -----------
          120 Treasury                                          1700
          130 Corporate Tax                                     1700
          140 Control And Credit                                1700
          150 Shareholder Services                              1700
          160 Benefits                                          1700
          170 Manufacturing                                     1700
          180 Construction                                      1700
          190 Contracting                                       1700
          200 Operations                                        1700
          210 IT Support                                        1700
          220 NOC                                               1700

DEPARTMENT_ID DEPARTMENT_NAME                MANAGER_ID LOCATION_ID
------------- ------------------------------ ---------- -----------
          230 IT Helpdesk                                       1700
          240 Government Sales                                  1700
          250 Retail Sales                                      1700
          260 Recruiting                                        1700
          270 Payroll                                           1700

27 rows selected.
```

Fig. 4.17. *Selecting all values from a table.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

All the columns in a table can be displayed by following the SELECT keyword with an asterisk (*). Instead of using * it can also be displayed by listing all the columns after the SELECT Keyword.

For example:

Select department_id, department_name, manager_id, location_id from departments;

4.8.3 Selecting Specific Columns

Syntax:

Select column_name1, column_name2….. from departments;

Example: Select department_id, department_name from departments;

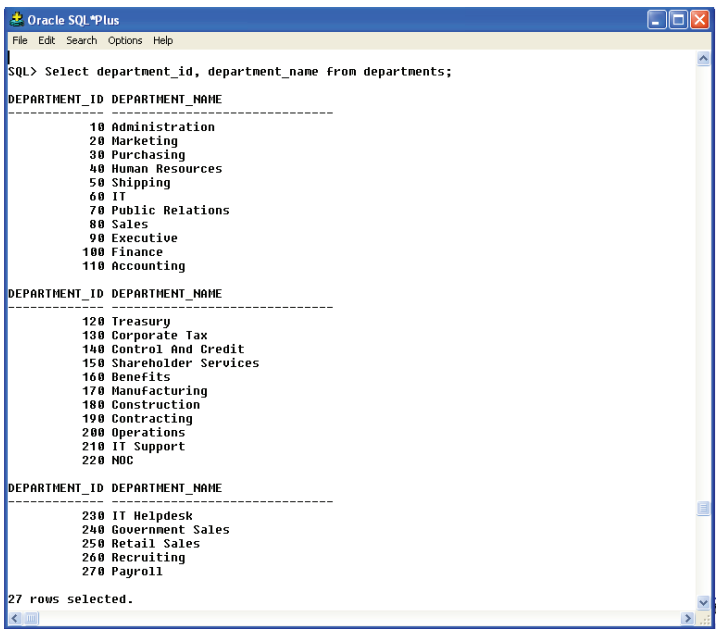


Fig. 4.18. *Selecting specific values from a table.*

Specific columns of the table can be selected by specifying the column name separated by commas in the SELECT statement. The order in which the column names are listed in the query will be the order in which the names will appear in the output. In the example given above the department_id and department_name are listed from the departments table.

4.8.4 Usage of DISTINCT Keyword in SELECT Statement

The SELECT DISTINCT statement is used to return only distinct (different) values. In a table, a column may contain many duplicate values; and sometimes you only want to list the different (distinct) values in such case DISTINCT keyword can be used.

Syntax:

SELECT DISTINCT column_name1, column_name2,… FROM table_name;

Example: select distinct location_id from departments;

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

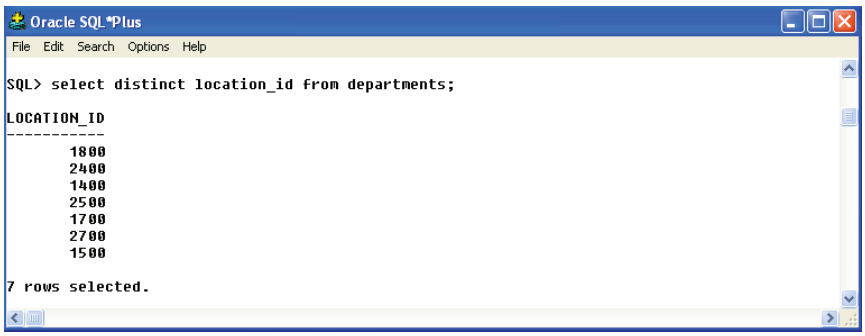


Fig. 4.19. *Usage of DISTINCT keyword in SELECT statement.*

4.8.5 Usage of ALL Keyword in SELECT Statement

The SELECT ALL statement is used to return all the values. In a table, a column may contain many duplicate values and if you want to list all the values without caring about the duplicate values in such cases ALL keyword can be used. If SELECT statement will use ALL keyword by default even if ALL keyword is not specified explicitly. If duplicate is to be removed then DISTINCT key should be explicitly used in the SELECT statement.

Syntax:

SELECT ALL column_name1, column_name2,… FROM table_name;

Example: select all location_id from departments;

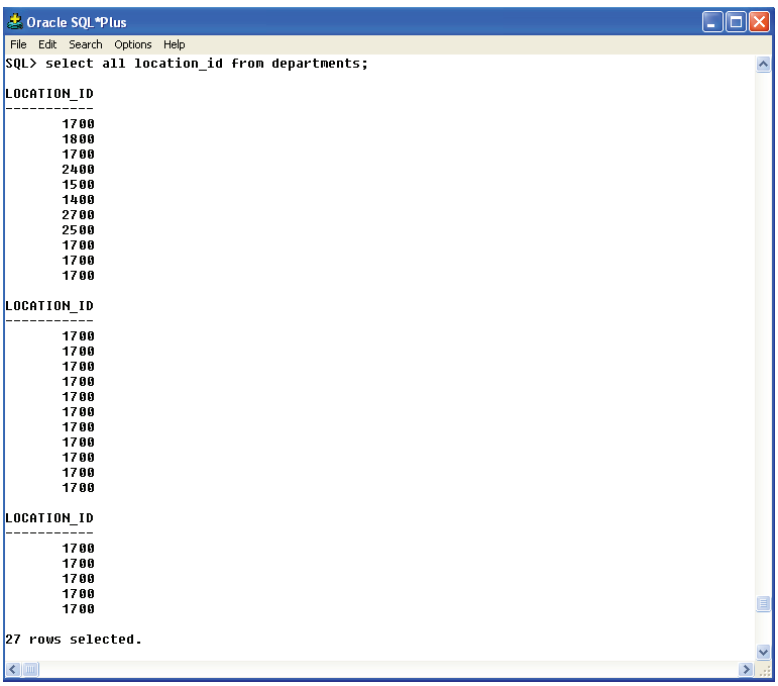


Fig. 4.20. *Usage of ALL keyword in SELECT statement.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.9 COLUMN ALIAS NAME

Column Alias is nothing but renaming the columns. The SQL provides a mechanism for renaming both relations and attributes. It uses the as clause using the form:

Old-name as new-name

The keyword 'as' is optional. The result of the query is same whether the AS keyword is used or not. Even if the column heading is given in lowercase it to be in uppercase by default when the query displays the result. The alias name should immediately follow the column name. The alias name should be specified in double quotation if it contains spaces or special character or if it is case sensitive.

Syntax:

SELECT old column_name1 AS new column_name1, old column_name2 AS new column_name2,…… FROM table_name WHERE [condition];

Example: select department_id as d_id, department_name as d_name from departments;

The same query can be rewritten as

selectdepartment_id "d_id", department_name "d_name" from departments;

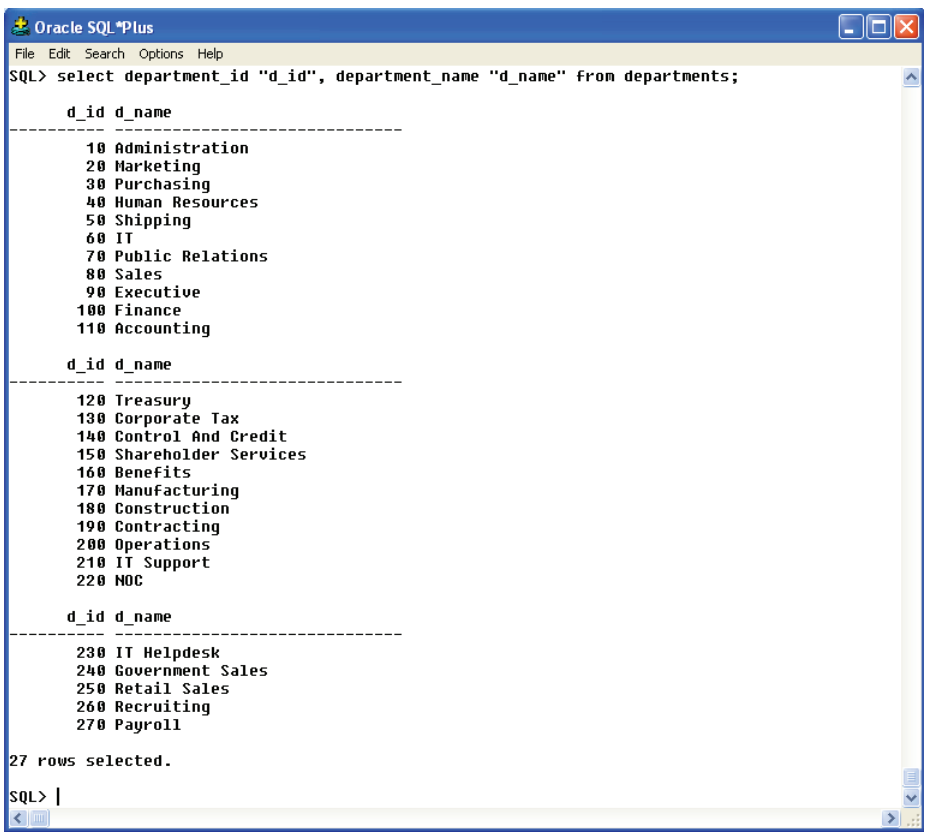


Fig. 4.21. *Query using alias name for columns in the SELECT statement.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.10 STRING OPERATION

SQL includes a string-matching operator for comparisons on character strings. The operator "like" uses patterns that are described using two special characters:

- The percent sign (%) matches any substring
- The underscore (_) matches any character

The percent sign represents zero, one, or multiple characters. The underscore represents a single number or character. The symbols can be used in combinations.

Syntax:

SELECT FROM table_name WHERE column LIKE 'XXXX%'

or

SELECT FROM table_name WHERE column LIKE '%XXXX%'

or

SELECT FROM table_name WHERE column LIKE 'XXXX_'

or

SELECT FROM table_name WHERE column LIKE '_XXXX'

or

SELECT FROM table_name WHERE column LIKE '_XXXX_'

You can combine N number of conditions using AND or OR operators. Here, XXXX could be any numeric or string value.

Example: Here are number of examples showing WHERE part having different LIKE clause with '%' and '_' operators:

Statement	*Description*
WHERE LOCATION_ID LIKE '200%'	Finds any values that start with 200
WHERE LOCATION_ID LIKE '%200%'	Finds any values that have 200 in any position
WHERE LOCATION_ID LIKE '_00%'	Finds any values that have 00 in the second and third positions
WHERE LOCATION_ID LIKE '2_%_%'	Finds any values that start with 2 and are at least 3 characters in length
WHERE LOCATION_ID LIKE '%2'	Finds any values that end with 2
WHERE LOCATION_ID LIKE '_2%3'	Finds any values that have a 2 in the second position and end with a 3
WHERE LOCATION_ID LIKE '2_ _ _3'	Finds any values in a five-digit number that start with 2 and end with 3

Example: select department_id,location_id from departments where location_id like'170%';

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

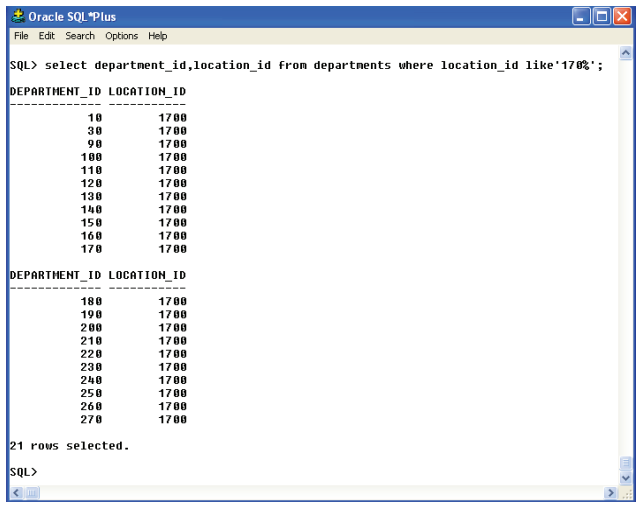


Fig. 4.22. *Query using string operation.*

4.11 CONCATENATION OPERATION

Concatenation operation links columns or character strings to other columns. It is represented by two vertical bars (||).

Syntax:

SELECT column_name1||column_name2 as new column_name from TABLE_NAME;

Example: select department_id||department_name as id_name from departments;

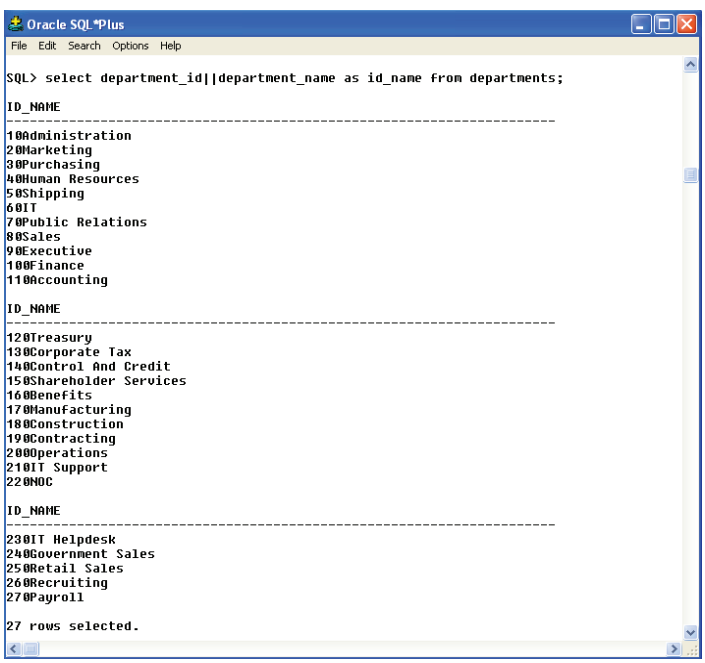


Fig. 4.23. *Query using concatenation operation.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

When the above query is executed the department_id and department_name of departments table is merged together and displayed under the column heading id_name.

Literals can also be included in the output. Literals are nothing but a character or number or date that is included in the SELECT list. Literal is not a column name or column alias. The literal will be printed once for each row. Date and character literals must be enclosed by single quotation mark (‘ ‘); number literals need not be enclosed in quotation.

Syntax:

*SELECT column_name1||**’literal’**||column_name2 as new column_name from TABLE_NAME;*

Example: select department_id||’corresponds to’||department_name as id_name from departments;

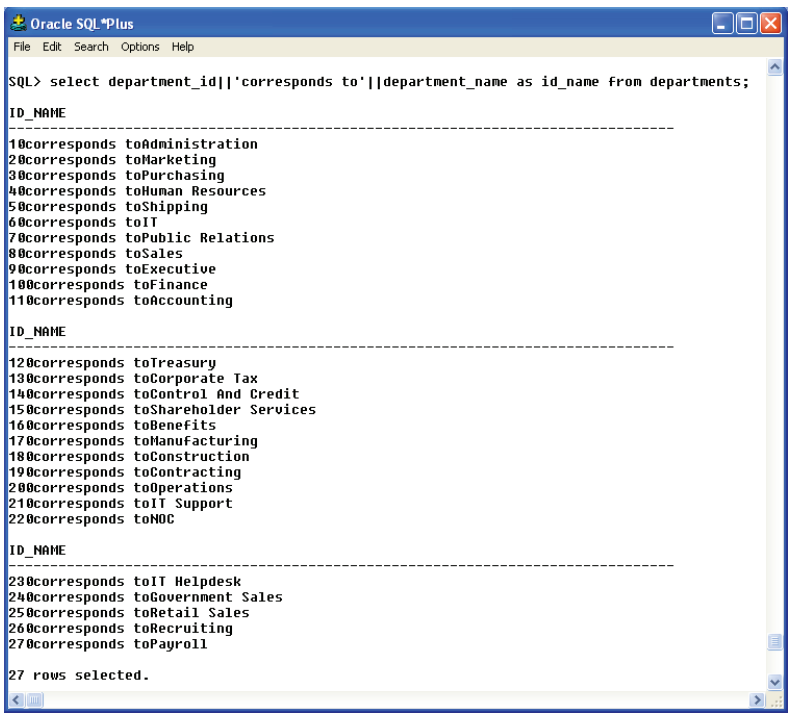


Fig. 4.24. *Query using concatenation operation along with literals.*

4.12 ORDERING THE DISPLAY OF TUPLES

The ORDER BY clause is used in a SELECT statement to sort results either in ascending or descending order. Oracle sorts query results in ascending order by default.

Syntax:

SELECT column-list FROM table_name [WHERE condition] [ORDER BY column1, column2, .. columnN] [ASC | DESC];

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

ORDER BY – Specifies the order in which rows are displayed

ASC – Orders the rows in ascending order (this is the default order)

DESC – Orders the rows in descending order

The statement enclosed in square brackets [ ] is optional.

Example: select department_id, department_name from departments order by department_id desc;

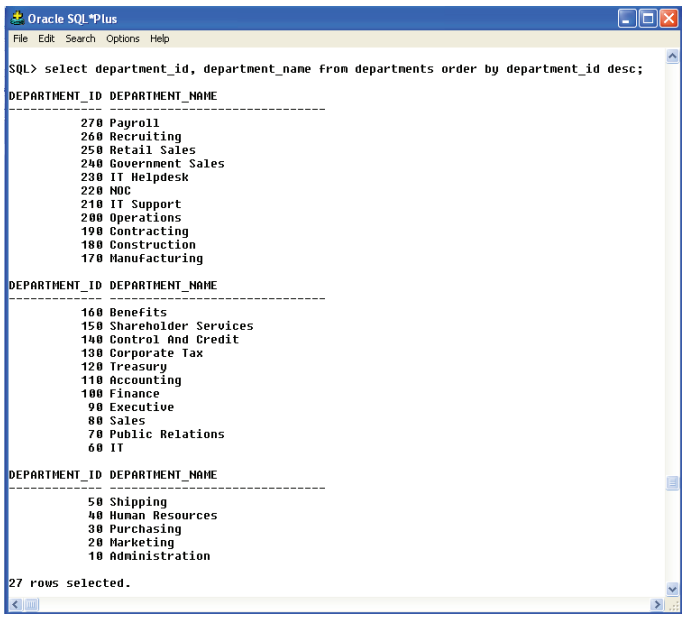


Fig. 4.25. *Ordering the tuples in descending order.*

4.13 SET OPERATIONS

Set operators combine the results of two queries into a single one. Let us consider the following tables to learn about set operations.

EMPLOYEES TABLE

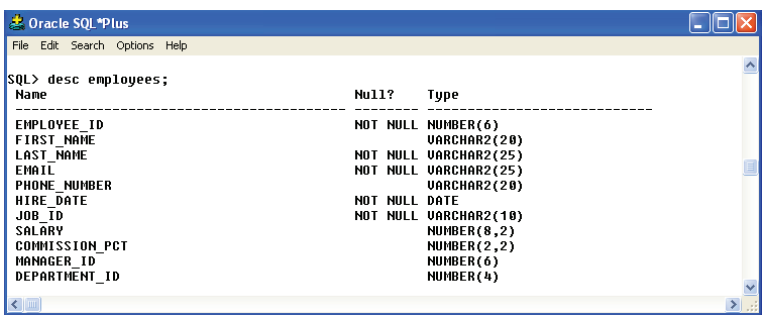


Fig. 4.26. *Employees table.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

JOB_HISTORY TABLE

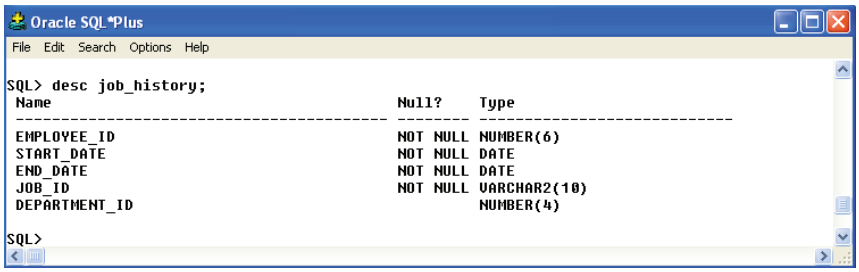


Fig. 4.27. *Job_History table.*

4.13.1 Union

The SQL **UNION** clause/operator is used to combine the results of two or more SELECT statements without returning any duplicate rows. To use UNION, each SELECT must have the same number of columns selected, the same number of column expressions, the same data type, and have them in the same order, but they do not have to be the same length.

Syntax:

SELECT column1 [, column2] FROM table1 [, table2] [WHERE condition]

UNION

SELECT column1 [, column2] FROM table1 [, table2] [WHERE condition]

Example: select employee_id, job_id from employees **union** select employee_id, job_id from job_history;

To know the current and previous job details of all employees the above query is executed.

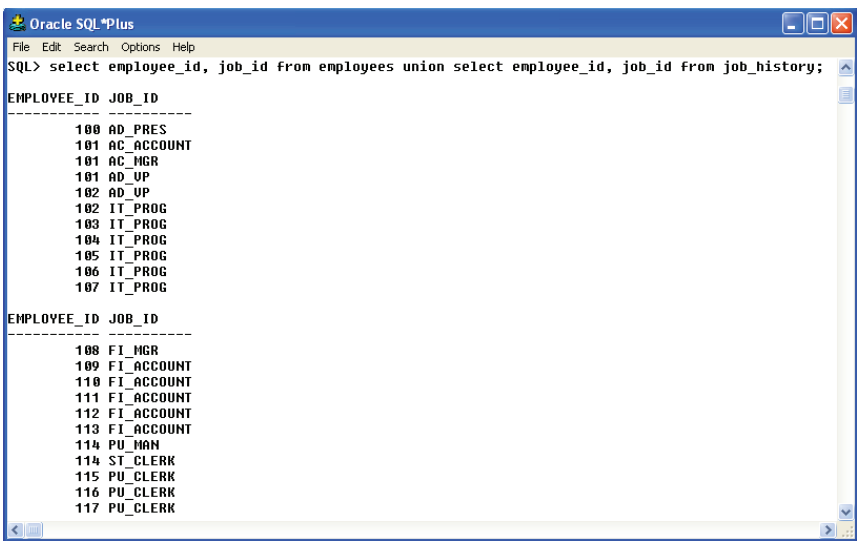


Fig. 4.28. *Union Operation of two tables.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.13.2 Union All

The UNION ALL operator is used to combine the results of two SELECT statements including duplicate rows. The same rules that apply to UNION apply to the UNION ALL operator. The output is not sorted by default and distinct keyword cannot be used with this operator.

Syntax:

SELECT column1 [, column2] FROM table1 [, table2] [WHERE condition]

UNION ALL

SELECT column1 [, column2] FROM table1 [, table2] [WHERE condition]

Example: select employee_id, job_id from employees **union all** select employee_id, job_id from job_history;

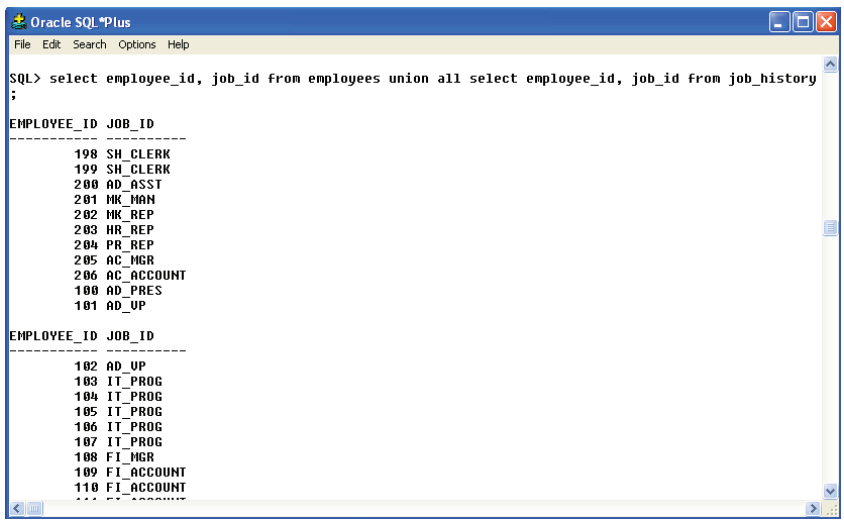


Fig. 4.29. *Union all of two tables.*

4.13.3 Intersect

The SQL **INTERSECT** clause/operator is used to combine two SELECT statements, but returns rows only from the first SELECT statement that are identical to a row in the second SELECT statement. This means INTERSECT returns only common rows returned by the two SELECT statements. Just as with the UNION operator, the same rules apply when using the INTERSECT operator.

Syntax:

SELECT column1 [, column2] FROM table1 [, table2] [WHERE condition]

INTERSECT

SELECT column1 [, column2] FROM table1 [, table2] [WHERE condition]

Example: select employee_id, job_id from employees **intersect** select employee_id, job_id from job_history;

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

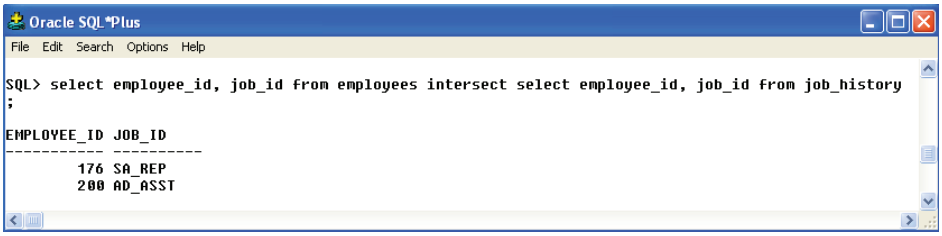


Fig. 4.30. *Intersection all of two tables.*

4.13.4 Minus or Except

The SQL Minus or **EXCEPT** clause/operator is used to combine two SELECT statements and returns rows from the first SELECT statement that are not returned by the second SELECT statement. This means EXCEPT returns only rows, which are not available in second SELECT statement. EXCEPT operator follows the same rules as the UNION operator.

Syntax:

SELECT column1 [, column2] FROM table1 [, table2] [WHERE condition]

EXCEPT

SELECT column1 [, column2] FROM table1 [, table2] [WHERE condition]

Example: select employee_id from employees **minus** select employee_id from job_history;

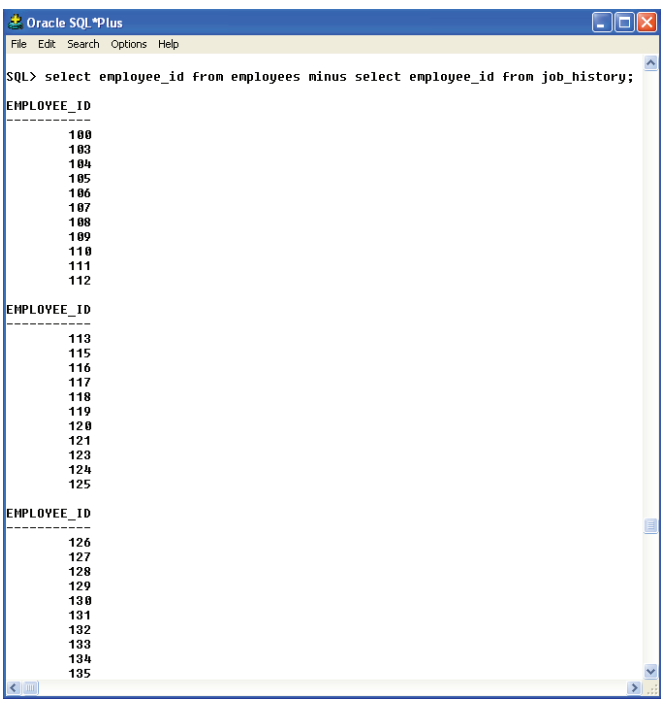


Fig. 4.31. *Minus all of two tables.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.14 WHERE CLAUSE

The WHERE Clause is used to retrieve specific information from a table excluding other irrelevant data.

For example, when you want to see the information of all employees who work in a particular department we can remove the irrelevant information of other employees using where clause. This helps to reduce the processing time. So SQL offers a feature called WHERE clause, which we can use to restrict the data that is retrieved. The condition you provide in the WHERE clause filters the rows retrieved from the table and gives you only those rows which you expected to see. WHERE clause can be used along with SELECT, DELETE and UPDATE statements.

Syntax:

SELECT column_list FROM table-name WHERE condition;

Example: select employee_id from employees where department_id=50;

This query filters and displays the employee_id of those employees who belongs to department with department_id=50.

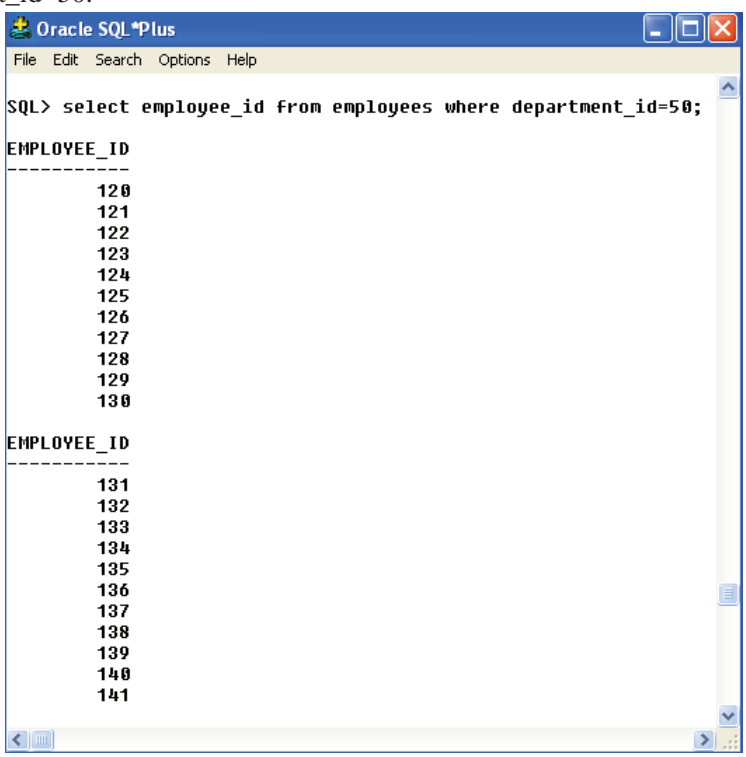


Fig. 4.32. *Query using WHERE clause for restricting column retrieval using numerical value.*

Character strings and date values can also be used in where clause but should be enclosed in single quotation marks. Character values are case sensitive and date values are format sensitive.

Example: select last_name, department_id from employees where last_name='Whalen';

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

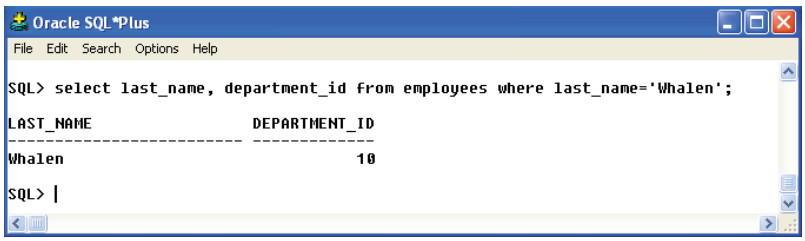


Fig. 4.33. *Query using WHERE clause for restricting column retrieval using character value.*

Comparison Operators and Logical Operators are used in WHERE Clause. Aliases defined for the columns in the SELECT statement cannot be used in the WHERE clause to set conditions. Only aliases created for tables can be used to reference the columns in the table. Expressions can also be used in the WHERE clause of the SELECT statement.

4.14.1 Operators in the WHERE Clause

The following operators can be used in the WHERE clause:

Operator	*Description*
=	Equal
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
BETWEEN	Between an inclusive range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

***Example* 1:** select last_name, salary from employees where salary<3000;

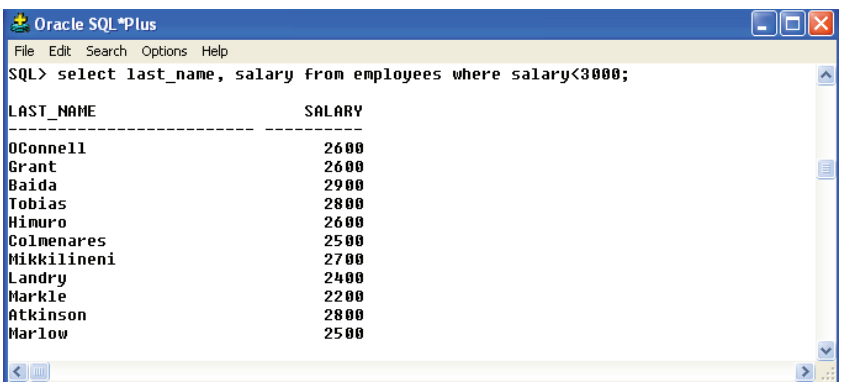


Fig. 4.34. *Query using operators in WHERE clause.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

***Example* 2: (Between operator)** select last_name, salary from employees where salary between 2500 and 3000;

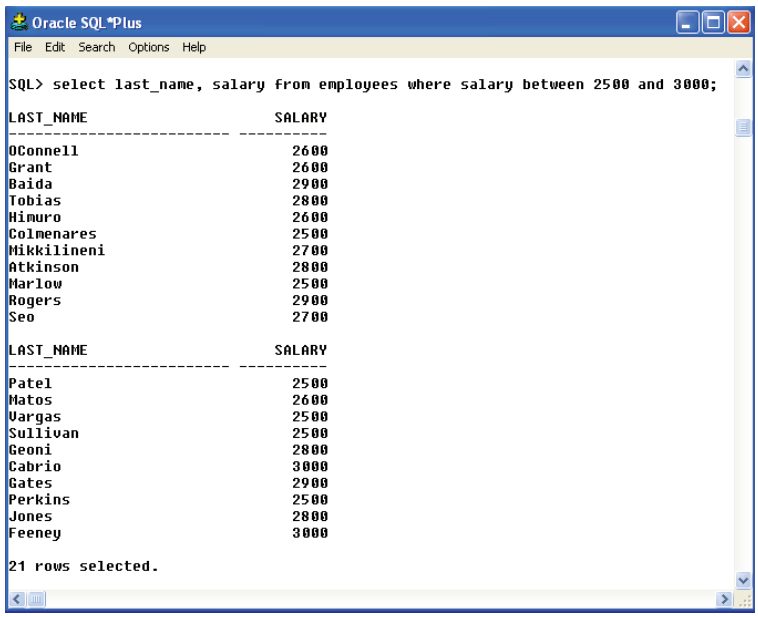


Fig. 4.35. *Query using BETWEEN operators using numerical values in WHERE clause.*

***Example* 3: (Between operator)** select last_name from employees where last_name between 'King' and 'Smith';

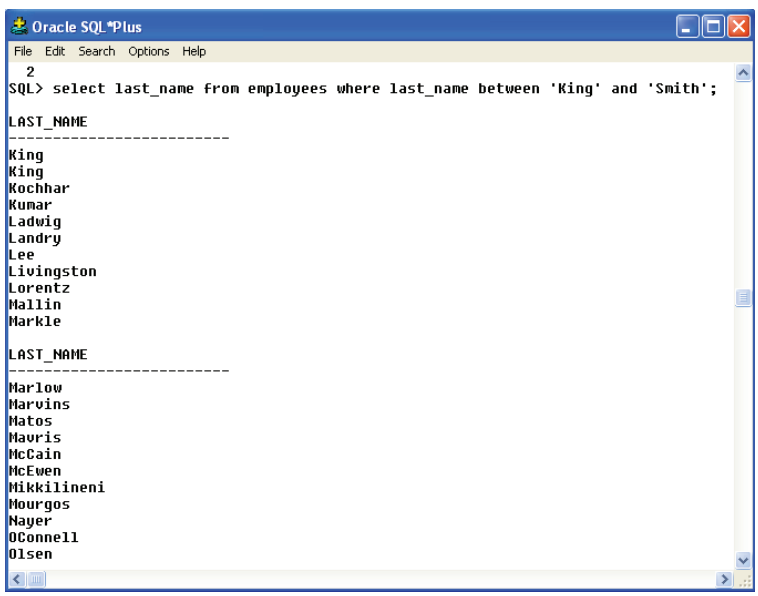


Fig. 4.36. *Query using BETWEEN operators using character values in WHERE clause.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

***Example* 4: (IN) Select employee_id, last_name, salary from employees where manager_id in(100,101,201);**

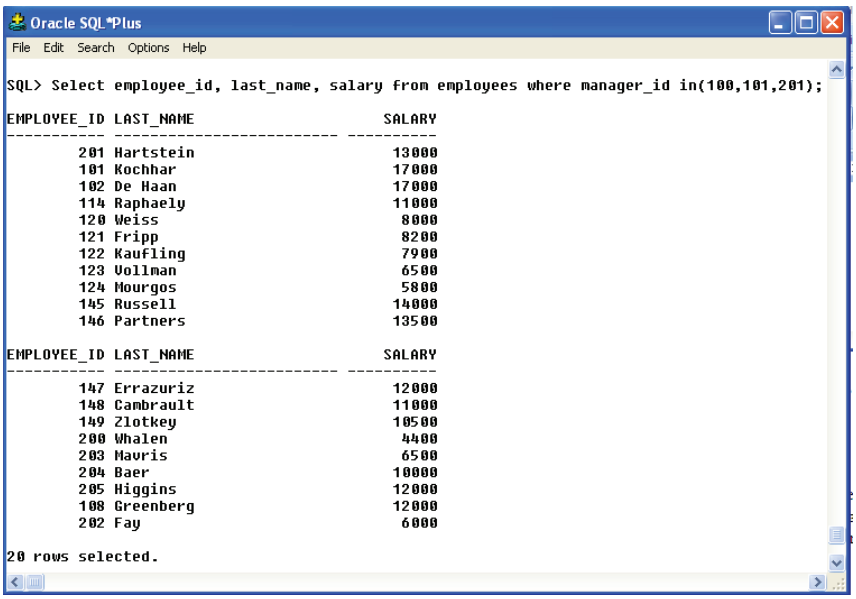


Fig. 4.37. *Query using IN operators in WHERE clause.*

4.15 OPERATORS

An operator is a reserved word or a character used primarily in an SQL statement's WHERE clause to perform operation(s), such as comparisons and arithmetic operations. Operators are used to specify conditions in an SQL statement and to serve as conjunctions for multiple conditions in a statement.

- Arithmetic operators
- Comparison operators
- Logical operators
- Operators used to negate conditions

4.15.1 Arithmetic Operators

Operator	*Description*
+	Addition - Adds values on either side of the operator
-	Subtraction - Subtracts right hand operand from left hand operand
*	Multiplication - Multiplies values on either side of the operator
/	Division - Divides left hand operand by right hand operand
%	Modulus - Divides left hand operand by right hand operand and returns remainder

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.15.2. SQL Comparison Operators

Operator	*Description*
=	Checks if the values of two operands are equal or not, if yes then condition becomes true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.
<>	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.
!<	Checks if the value of left operand is not less than the value of right operand, if yes then condition becomes true.
!>	Checks if the value of left operand is not greater than the value of right operand, if yes then condition becomes true.

4.15.3 SQL Logical Operators

Operator	*Description*
ALL	The ALL operator is used to compare a value to all values in another value set.
AND	The AND operator allows the existence of multiple conditions in an SQL statement's WHERE clause.
ANY	The ANY operator is used to compare a value to any applicable value in the list according to the condition.
BETWEEN	The BETWEEN operator is used to search for values that are within a set of values, given the minimum value and the maximum value.
EXISTS	The EXISTS operator is used to search for the presence of a row in a specified table that meets certain criteria.
IN	The IN operator is used to compare a value to a list of literal values that have been specified.
LIKE	The LIKE operator is used to compare a value to similar values using wildcard operators.
NOT	The NOT operator reverses the meaning of the logical operator with which it is used. Eg: NOT EXISTS, NOT BETWEEN, NOT IN, etc. **This is a negate operator.**
OR	The OR operator is used to combine multiple conditions in an SQL statement's WHERE clause.
IS NULL	The NULL operator is used to compare a value with a NULL value.
UNIQUE	The UNIQUE operator searches every row of a specified table for uniqueness (no duplicates).

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.16 AGGREGATE FUNCTION

Aggregate functions perform a calculation on a set of values and return a single value. Except for COUNT, aggregate functions ignore null values. Aggregate functions are frequently used with the GROUP BY clause of the SELECT statement. All aggregate functions are deterministic. This means aggregate functions return the same value any time that they are called by using a specific set of input values.

These functions operate on the multiset of values of a column of a relation, and return a value

Consider the following employees table

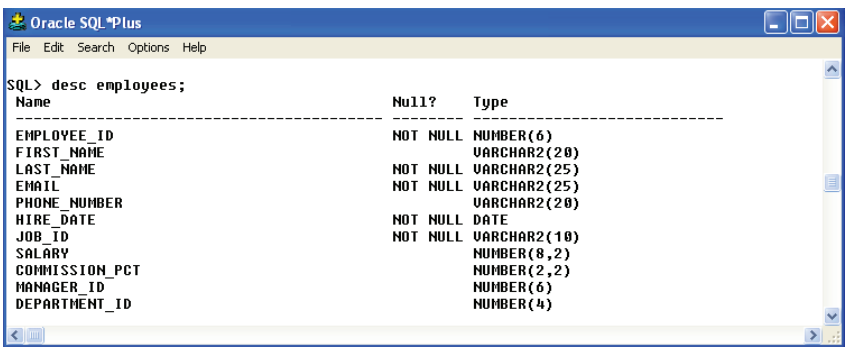


Fig. 4.38. *Employees table.*

4.16.1 AVG

The AVG() function returns the average value of a numeric column.

Syntax: *SELECT AVG(column_name) FROM table_name*

***Example*:** Find the average of salary from the employees table

Query: select **avg** (salary) from employees;

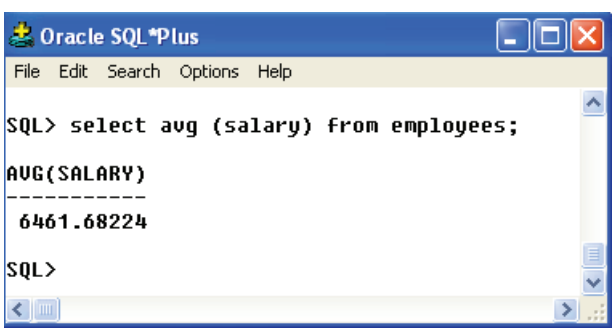


Fig. 4.39. *Query using AVG() function.*

4.16.2 SUM

The SUM() function returns the total sum of a numeric column.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Syntax: *SELECT SUM(column_name) FROM table_name;*

***Example*:** Find the sum of salary from the employees table

Query: select **sum** (salary) from employees;

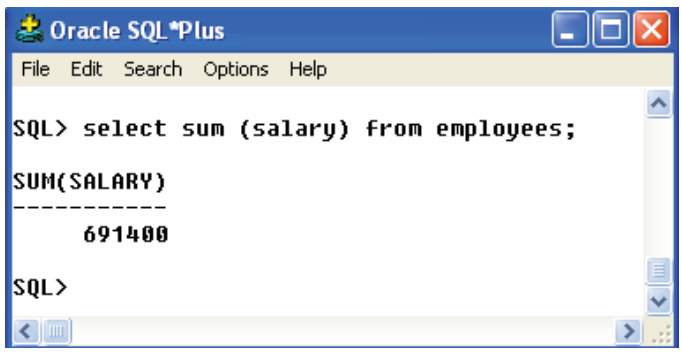


Fig. 4.40. *Query using SUM() function.*

4.16.3 MAX

The MAX() function returns the largest value of the selected column.

Syntax: *SELECT MAX(column_name) FROM table_name;*

Example: Find the Maximum value of account balance from the account table.

Query: select**max**(salary) from employees;

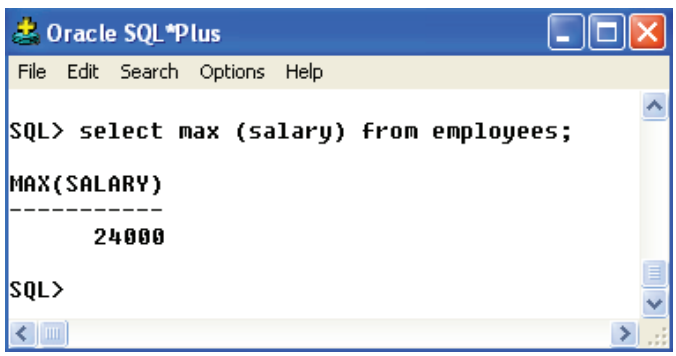


Fig. 4.41. *Query using MAX() function.*

4.16.4 MIN

The MIN() function returns the smallest value of the selected column.

Syntax: *SELECT MIN(column_name) FROM table_name;*

***Example*:** Find the Minimum value of account balance from the account table.

Query: select**min**(salary) from employees;

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

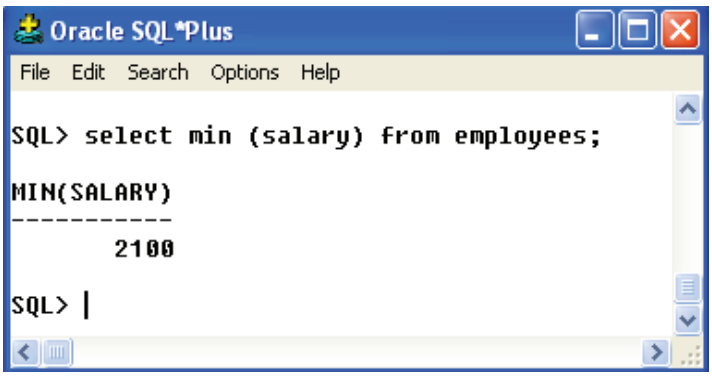


Fig. 4.42. *Query using MIN() function.*

4.16.5 COUNT

The COUNT() function returns the number of rows that matches a specified criteria.

Syntax: *SELECT COUNT(column_name) FROM table_name;*

***Example*:** Find the number rows in the employees table.

Query: select**count (*)** from employees;

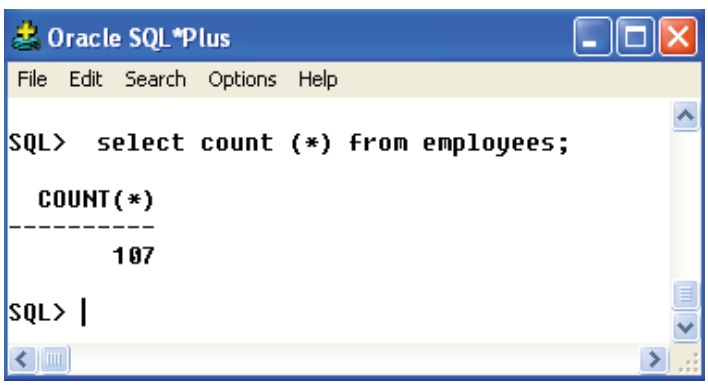


Fig. 4.43. *Query using COUNT() function.*

4.16.6 COUNT*

The COUNT(*) function returns the number of records in a table.

Syntax: *SELECT COUNT(*) FROM table_name;*

***Example*:** Find the number of rows in the salary column of employees table.

Query: select **count (salary)** from employees;

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

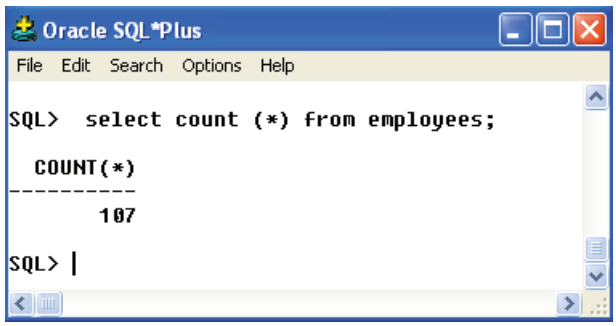


Fig. 4.43. *Query using COUNT*() function.*

4.17 GROUP BY CLAUSE

The SQL **GROUP BY** clause is used in collaboration with the SELECT statement to arrange identical data into groups. The GROUP BY clause follows the WHERE clause in a SELECT statement and precedes the ORDER BY clause.

In group function WHERE clause can be used to exclude rows before dividing them into groups. GROUP BY clause should include column names and column alias name cannot be included in the GROUP BY clause. The group by clause should contain all the columns in the select list expect those used along with the group functions.

Syntax:

SELECT column1, column2 FROM table_name WHERE [conditions]
[GROUP BY column1, column2]
[ORDER BY column1, column2]

Example: Select department_id, AVG(salary) from employees group by department_id;

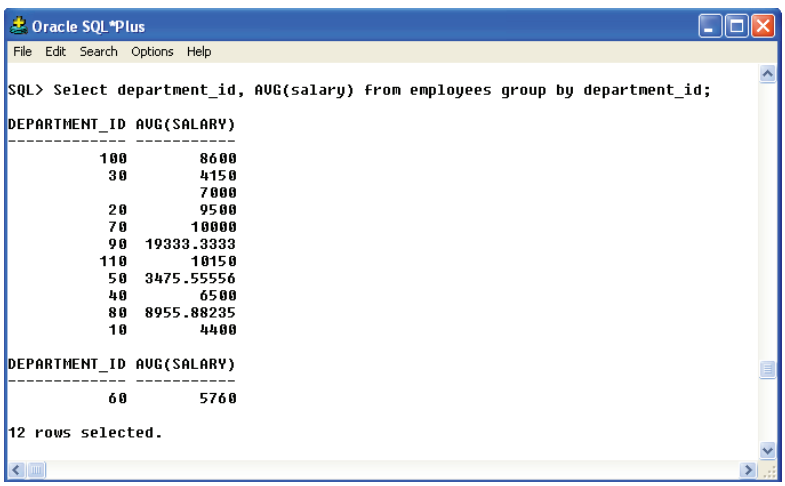


Fig. 4.44. *SELECT statement with GROUP BY clause.*

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.18 HAVING CLAUSE

The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions. Having clause is used to filter data based on the group functions. This is similar to WHERE condition but is used with group functions. Group functions cannot be used in WHERE Clause but can be used in HAVING clause.

If you want to select the department that has total salary paid for its employees more than 25000, the sql query would be like;

Query: select department_id, sum(salary) from employees group by department_id having sum(salary)>25000;

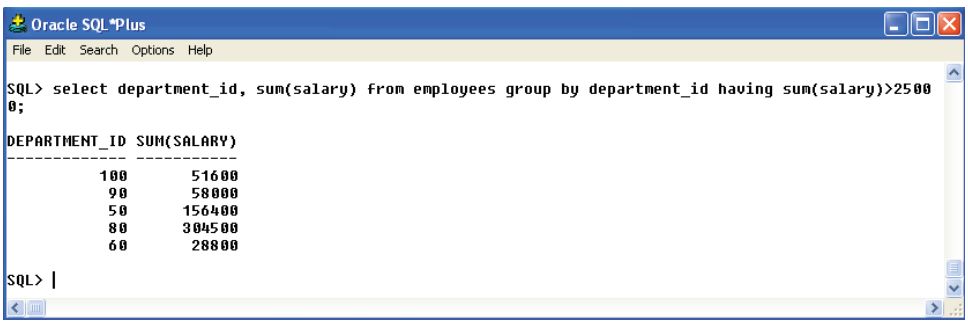


Fig. 4.45. *SELECT statement with HAVING clause.*

4.19 NESTED SUBQUERIES

A Subquery or Inner query or Nested query is a query within another SQL query and embedded within the WHERE clause. A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.

Subqueries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN etc.

There are a few rules that subqueries must follow:

- Subqueries must be enclosed within parentheses.
- A subquery can have only one column in the SELECT clause, unless multiple columns are in the main query for the subquery to compare its selected columns.
- An ORDER BY cannot be used in a subquery, although the main query can use an ORDER BY. The GROUP BY can be used to perform the same function as the ORDER BY in a subquery.
- Subqueries that return more than one row can only be used with multiple value operators, such as the IN operator.
- A subquery cannot be immediately enclosed in a set function.
- The BETWEEN operator cannot be used with a subquery; however, the BETWEEN operator can be used within the subquery.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.19.1 Subqueries with the SELECT Statement:

Subqueries are most frequently used with the SELECT statement. The basic syntax is as follows:

SELECT column_name [, column_name]
FROM table1 [, table2]
WHERE column_name OPERATOR
(SELECT column_name [, column_name]
FROM table1 [, table2]
[WHERE])

Example: Consider the employees table

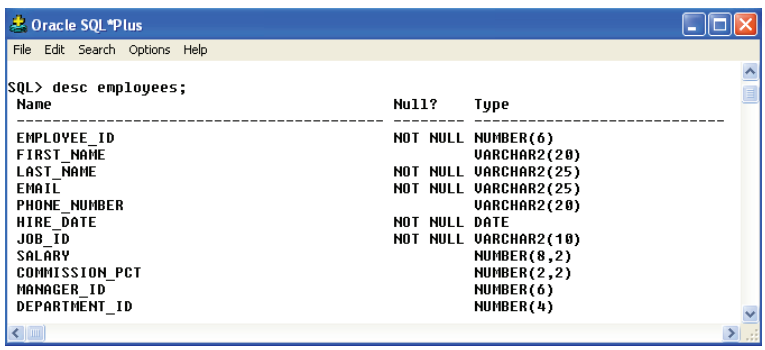


Fig. 4.46. *Employees table.*

Query: select * from employees where employee_id in (select employee_id from employees where salary >4500);

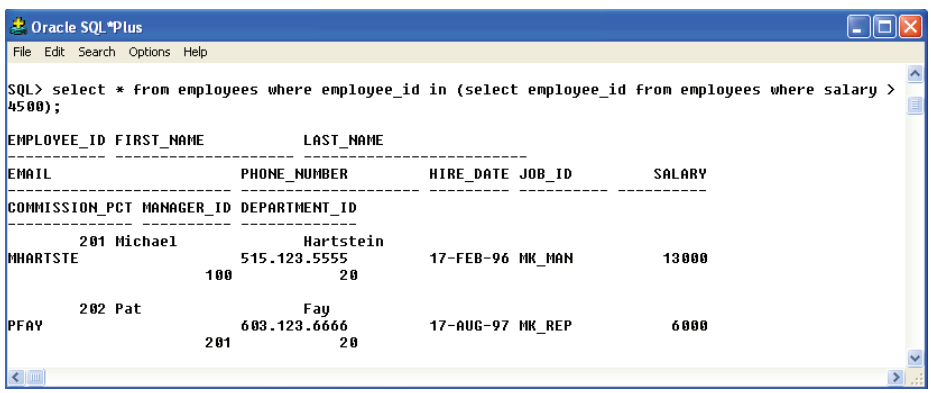


Fig. 4.47. *Query depicting subquery concept.*

4.19.2 Subqueries with the INSERT Statement

Subqueries also can be used with INSERT statements. The INSERT statement uses the data returned from the subquery to insert into another table. The selected data in the subquery can be modified

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

with any of the character, date or number functions. When adding a new row, you should ensure the datatype of the value and the column matches. The integrity constraints defined for the table should not be affected.

The basic syntax is as follows:

INSERT INTO table_name [(column1 [, column2])]
SELECT [|column1 [, column2]*
FROM table1 [, table2]
[WHERE VALUE OPERATOR]

Example: Consider a table EMPLOYEES1 with two fields employee_id and department_id. Now to copy the values of employee_id and department_id from EMPLOYEES table into EMPLOYEES1, following query is executed:

Before executing the query create a table EMPLOYEES1 with CREATE table statement and then insert the values.

Query: insert into employees1(employee_id, department_id) select employee_id, department_id from employees;

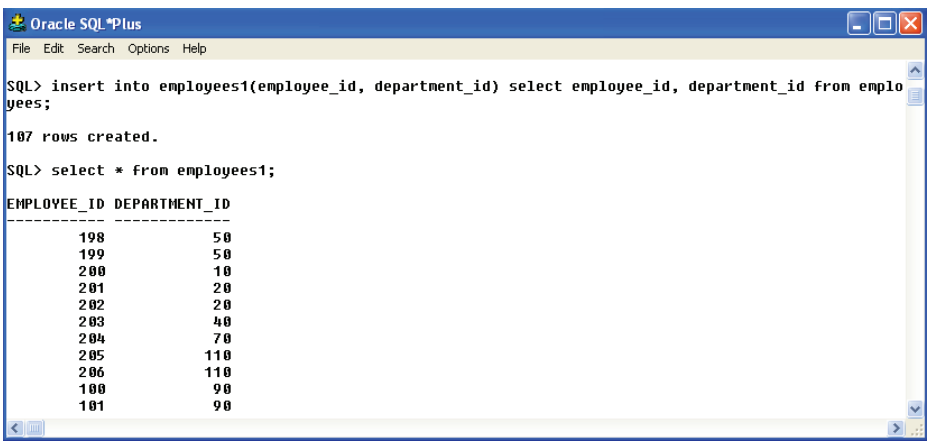


Fig. 4.48. *Subquery with INSERT statement.*

4.19.3 Subqueries with the UPDATE Statement

The subquery can be used with the UPDATE statement. Either single or multiple columns in a table can be updated when using a subquery with the UPDATE statement.

The basic syntax is as follows:

UPDATE table
SET column_name = new_value
[WHERE OPERATOR [VALUE]
(SELECT COLUMN_NAME
FROM TABLE_NAME)
[WHERE)]

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Example:

Query: update employees set salary = salary *0.25 where department_id in (select department_id from employees);

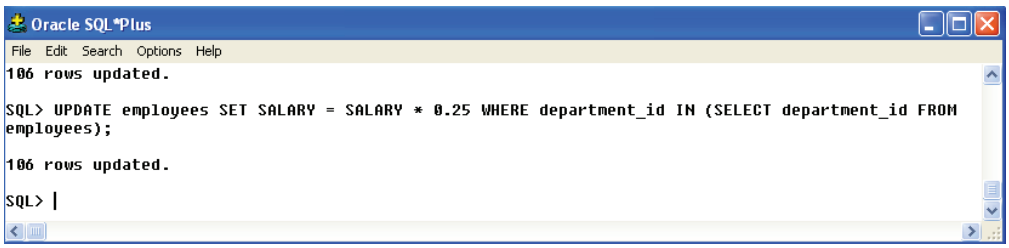


Fig. 4.49. *Subquery with UPDATE statement.*

Query: update employees set salary = salary *0.25 where department_id in (select department_id from employees where department_id=50);

This query is same as the previous but checks for a condition in the inner query.

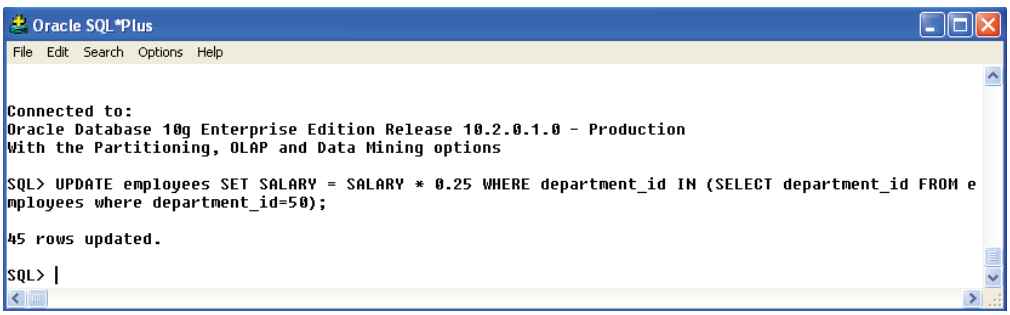


Fig. 4.50. *Subquery with UPDATE statement with condition specified in WHERE clause.*

4.19.4 Subqueries with the DELETE Statement

The subquery can be used in conjunction with the DELETE statement like with any other statements mentioned above.

The basic syntax is as follows:

DELETE FROM TABLE_NAME

[WHERE OPERATOR [VALUE]

(SELECT COLUMN_NAME

FROM TABLE_NAME)

[WHERE)]

Example: delete from employees1 where employee_id in (select employee_id from employees);

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

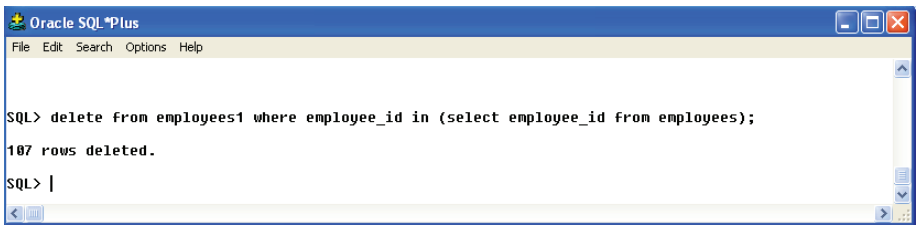


Fig. 4.51. *Subquery with DELETE statement.*

4.20 NULL VALUES

NULL values represent missing unknown data. By default, a table column can hold NULL values. If a column in a table is optional, we can insert a new record or update an existing record without adding a value to this column. This means that the field will be saved with a NULL value.

NULL values are treated differently from other values. NULL is used as a placeholder for unknown or inapplicable or unavailable or unassigned values. It is not possible to compare NULL and 0; they are not equivalent.

4.21 DATABASE OBJECTS

A database object in a relational database is a data structure used to either store or reference data. The most common object that most people interact with is the table.

The database objects include:

1. Tables
2. Views
3. Sequence
4. Triggers
5. Indexes

4.21.1 Tables

Table is a relation that is used to store records of related data. It is a logical structure maintained by the database manager. It is made up of columns and rows. At the intersection of every column and row there is a specific data item called a value or field. A base table is created with the CREATE TABLE statement and is used to hold persistent user data. All queries can be performed in a Table.

Creating a basic table involves naming the table and defining its columns and each column's data type. The SQL **CREATE TABLE** statement is used to create a new table.

Syntax: Basic syntax of CREATE TABLE statement is as follows:

```
CREATE TABLE table_name(
column1 datatype,
column2 datatype,
column3 datatype,
    .....
columnN datatype,
PRIMARY KEY( one or more columns )
);
```

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

CREATE TABLE is the keyword telling the database system what you want to do. In this case, you want to create a new table. The unique name or identifier for the table follows the CREATE TABLE statement. Then in brackets comes the list defining each column in the table and what sort of data type it is. The syntax becomes clearer with an example below. A copy of an existing table can be created using a combination of the CREATE TABLE statement and the SELECT statement.

Example: Following is an example, which creates a CUSTOMERS table with ID as primary key and NOT NULL are the constraints showing that these fields cannot be NULL while creating records in this table:

```
SQL> CREATE TABLE CUSTOMERS (
     ID   INT              NOT NULL,
     NAME VARCHAR (20)     NOT NULL,
     AGE INT              NOT NULL,
     ADDRESS CHAR (25) ,
     SALARY   DECIMAL (18, 2),
     PRIMARY KEY (ID)
     );
```

The success of the created table can be verified by looking at the message displayed by the SQL server, otherwise you can use **DESC** command as follows:

```
SQL> DESC CUSTOMERS;
```

Field	*Type*	*Null*	*Key*	*Default*	*Extra*
ID	int(11)	NO	PRI		
NAME	varchar(20)	NO			
AGE	int(11)	NO			
ADDRESS	char(25)	YES		NULL	
SALARY	decimal(18,2)	YES		NULL	

Now, the CUSTOMERS table is available in the database which you can used to store required information related to customers.

4.21.2 Views

VIEW is a stored SQL query used as a "Virtual table" that logically represents subsets of data from one or more tables. It provides an alternative way of looking at the data in one or more tables. A view is a logical table based on a table or another view. The table on which view is based is called a base table.

It is a named specification of a result table. The specification is a SELECT statement that is executed whenever the view is referenced in an SQL statement. Consider a view to have columns and rows just like a base table. For retrieval, all views can be used just like base tables.

When the column of a view is directly derived from the column of a base table, that column inherits any constraints that apply to the column of the base table. For example, if a view includes a foreign key of its base table, INSERT and UPDATE operations using that view are subject to

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

the same referential constraints as the base table. Also, if the base table of a view is a parent table, DELETE and UPDATE operations using that view are subject to the same rule as DELETE and UPDATE operations on the base table.

4.21.2.1 Advantages of View

- **Restricts data access:** views restrict access to the data because the view can display selected columns from the table.
- **Makes complex query easy:** view can be used to make simple queries to retrieve the results of complicated queries. View can be used to query information from multiple tables without the user knowing how to write a join statement.
- **To present different views of the same data:** view provides data independence for ad-hoc users and application programs. One view can be used to retrieve data from several tables.
- **To provide data independence:** Views provide groups of users' access to data according to their particular criteria.

4.21.2.2 Creating Views

Database views are created using the **CREATE VIEW** statement. Views can be created from a single table, multiple tables, or another view.

To create a view, a user must have the appropriate system privilege according to the specific implementation.

The basic CREATE VIEW syntax is as follows:

```
CREATE VIEW view_name AS
SELECT column_name(s)
FROM table_name
WHERE condition
```

Example: Consider the CUSTOMERS table having the following records:

Id	*Name*	*Age*	*Address*	*Salary*
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Query to create View

```
SQL > CREATE VIEW CUSTOMERS_VIEW AS
SELECT name, age
FROM CUSTOMERS;
```

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

When the above given query is executed View will be created in the name CUSTOMERS_VIEW. Now, we can query CUSTOMERS_VIEW in similar way as you query an actual table.

SQL > SELECT * FROM CUSTOMERS_VIEW;

This query would produce the following output

Name	*Age*
Ramesh	32
Khilan	25
Kaushik	23
Chaitali	25
Hardik	27
Komal	22
Muffy	24

4.21.2.3 The WITH CHECK OPTION

The WITH CHECK OPTION is a CREATE VIEW statement option. The purpose of the WITH CHECK OPTION is to ensure that all UPDATE and INSERTs satisfy the condition(s) in the view definition. If they do not satisfy the condition(s), the UPDATE or INSERT returns an error.

The following is an example of creating same view CUSTOMERS_VIEW with the WITH CHECK OPTION:

```
CREATE VIEW CUSTOMERS_VIEW AS
SELECT name, age
FROM CUSTOMERS
WHERE age IS NOT NULL
WITH CHECK OPTION;
```

The WITH CHECK OPTION in this case should disallow the entry of any NULL values in the view's AGE column, because the view is defined by data that does not have a NULL value in the AGE column.

4.21.2.4 Updating a View

A view can be updated under certain conditions:

- The SELECT clause may not contain the keyword DISTINCT.
- The SELECT clause may not contain summary functions.
- The SELECT clause may not contain set functions.
- The SELECT clause may not contain set operators.
- The SELECT clause may not contain an ORDER BY clause.
- The FROM clause may not contain multiple tables.
- The WHERE clause may not contain subqueries.
- The query may not contain GROUP BY or HAVING.
- Calculated columns may not be updated.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

- All NOT NULL columns from the base table must be included in the view in order for the INSERT query to function.

So if a view satisfies all the above-mentioned rules then you can update a view. Following is an example to update the age of Ramesh:

```
SQL > UPDATE CUSTOMERS_VIEW
SET AGE =35
WHERE name='Ramesh';
```

This would ultimately update the base table CUSTOMERS and same would reflect in the view itself. Now, try to query base table, and SELECT statement would produce the following result:

Id	*Name*	*Age*	*Address*	*Salary*
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

4.21.2.5 Inserting Rows into a View

Rows of data can be inserted into a view. The same rules that apply to the UPDATE command also apply to the INSERT command. Here we cannot insert rows in CUSTOMERS_VIEW because we have not included all the NOT NULL columns in this view, otherwise you can insert rows in a view in similar way as you insert them in a table.

4.21.2.6 Deleting Rows into a View

Rows of data can be deleted from a view. The same rules that apply to the UPDATE and INSERT commands apply to the DELETE command.

Following is an example to delete a record having AGE= 22.

```
SQL > DELETE FROM CUSTOMERS_VIEW
WHERE age =22;
```

This would ultimately delete a row from the base table CUSTOMERS and same would reflect in the view itself. Now, try to query base table, and SELECT statement would produce the following result:

Id	*Name*	*Age*	*Address*	*Salary*
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Muffy	24	Indore	10000.00

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.21.2.7 Dropping Views

The view can be dropped if the view is no longer needed. The syntax is very simple as given below:

DROP VIEW view_name;

Following is an example to drop CUSTOMERS_VIEW from CUSTOMERS table:

DROP VIEW CUSTOMERS_VIEW;

When the above query is executed the view will be no longer present.

4.21.3 Sequences

A sequence is a database object that generates numbers in sequential order. Applications most often use these numbers when they require a unique value in a table such as primary key values. Some database management systems use an "auto number" concept or "auto increment" setting on numeric column types. Both the auto numbering columns and sequences provide a unique number in sequence used for a unique identifier. The following list describes the characteristics of sequences:

- Sequences are available to all users of the database
- Sequences are created using SQL statements
- Sequences have a minimum and maximum value (the defaults are minimum=0 and maximum=2^{63}-1); they can be dropped, but not reset
- Once a sequence returns a value, the sequence can never return that same value
- While sequence values are not tied to any particular table, a sequence is usually used to generate values for only one table
- Sequences increment by an amount specified when created (the default is 1)

4.21.3.1 Creating a Sequence

To create sequences, execute a CREATE SEQUENCE statement in the same way as an UPDATE or INSERT statement. The sequence information is stored in a data dictionary file in the same location as the rest of the data dictionary files for the database. If the data dictionary file does not exist, the SQL engine creates the file when it creates the first sequence.

The format for a CREATE SEQUENCE statement is as follows:

CREATE SEQUENCE sequence_name
[INCREMENT BY #]
[START WITH #]
[MAXVALUE # | NOMAXVALUE]
[MINVALUE # | NOMINVALUE]
[CYCLE | NOCYCLE]

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Variable	*Description*
INCREMENT BY	The increment value. This can be a positive or negative number.
START WITH	The start value for the sequence.
MAXVALUE	The maximum value that the sequence can generate. If specifying NOMAXVALUE, the maximum value is 2^{63}–1.
MINVALUE	The minimum value that the sequence can generate. If specifying NOMINVALUE, the minimum value is -2^{63}.
CYCLE	Specify CYCLE to indicate that when the maximum value is reached the sequence starts over again at the start value. Specify NOCYCLE to generate an error upon reaching the maximum value.

4.21.3.2 Dropping a Sequence

To drop a sequence, execute a DROP SEQUENCE statement. Use this function when a sequence is no longer useful, or to reset a sequence to an older number. To reset a sequence, first drop the sequence and then recreate it. Drop a sequence following this format:

Syntax: *DROP SEQUENCE sequence_name;*

Example: DROP SEQUENCE my_sequence

4.21.3.3 Using a Sequence

Use sequences when an application requires a unique identifier. INSERT statements, and occasionally UPDATE statements, are the most common places to use sequences. Two "functions" are available on sequences:

NEXTVAL: Returns the next value from the sequence.

CURVAL: Returns the value from the last call to NEXTVAL by the current user during the current connection. For example, if User A calls NEXTVAL and it returns 124, and User B immediately calls NEXTVAL getting 125, User A will get 124 when calling CURVAL, while User B will get 125 while calling CURVAL. It is important to understand the connection between the sequence value and a particular connection to the database. The user cannot call CURVAL until making a call to NEXTVAL at least once on the connection. CURVAL returns the current value returned from the sequence on the current connection, not the current value of the sequence.

Examples: To create the sequence:

CREATE SEQUENCE customer_seq INCREMENT BY 1 START WITH 100

To use the sequence to enter a record into the database:

INSERT INTO customer (cust_num, name, address)

VALUES (customer_seq.NEXTVAL,'John Doe','123 Main St.')

To find the value just entered into the database:

SELECT customer_seq.CURVAL AS LAST_CUST_NUM

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.21.4 Triggers

Triggers are stored programs, which are automatically executed or fired when some events occur or some condition is met. Triggers are written to be executed in response to any of the following events:

- A database manipulation (DML) statement (DELETE, INSERT, or UPDATE).
- A database definition (DDL) statement (CREATE, ALTER, or DROP).
- A database operation (SERVERERROR, LOGON, LOGOFF, STARTUP, or SHUTDOWN).

Triggers could be defined on the table, view, schema, or database with which the event is associated.

4.21.4.1 Benefits of Triggers

Triggers can be written for the following purposes:

- Generating some derived column values automatically
- Enforcing referential integrity
- Event logging and storing information on table access
- Auditing
- Synchronous replication of tables
- Imposing security authorizations
- Preventing invalid transactions

4.21.4.2 Creating Triggers

The syntax for creating a trigger is:

```
CREATE[OR REPLACE ]TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF}
{INSERT[OR]|UPDATE[OR]|DELETE}
[OF col_name]
ON table_name
[REFERENCING OLD AS o NEWAS n]
[FOR EACH ROW]
WHEN(condition)
DECLARE
        Declaration-statements
BEGIN
        Executable-statements
EXCEPTION
        Exception-handling-statements
END;
```

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Where,

- CREATE [OR REPLACE] TRIGGER trigger_name: Creates or replaces an existing trigger with the *trigger_name*.
- {BEFORE | AFTER | INSTEAD OF}: This specifies when the trigger would be executed. The INSTEAD OF clause is used for creating trigger on a view.
- {INSERT [OR] | UPDATE [OR] | DELETE}: This specifies the DML operation.
- [OF col_name]: This specifies the column name that would be updated.
- [ON table_name]: This specifies the name of the table associated with the trigger.
- [REFERENCING OLD AS NEW AS n]: This allows you to refer new and old values for various DML statements, like INSERT, UPDATE, and DELETE.
- [FOR EACH ROW]: This specifies a row level trigger, i.e., the trigger would be executed for each row being affected. Otherwise the trigger will execute just once when the SQL statement is executed, which is called a table level trigger.
- WHEN (condition): This provides a condition for the rows which the trigger would fire. This clause is valid only for row level triggers.

4.21.4.3 Types of Triggers

There are two types of triggers based on which level it is triggered.

1. **Row Level Trigger:** An event is triggered for each row updated, inserted or deleted.
2. **Statement Level Trigger:** An event is triggered for each SQL statement executed.

4.21.4.4 Trigger Execution Hierarchy

The following hierarchy is followed when a trigger is fired.

1. BEFORE statement trigger fires first.
2. Next BEFORE row level trigger fires, once for each row affected.
3. Then AFTER row level trigger fires once for each affected row. These events will alternates between BEFORE and AFTER row level triggers.
4. Finally the AFTER statement level trigger fires.

Example: To start with, we will be using the CUSTOMERS table below

Id	*Name*	*Age*	*Address*	*Salary*
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

The following program creates a **row level** trigger for the customers table that would fire for INSERT or UPDATE or DELETE operations performed on the CUSTOMERS table. This trigger will display the salary difference between the old values and new values:

```
CREATEOR REPLACE TRIGGER display_salary_changes
BEFORE DELETEORINSERTORUPDATEON customers
FOR EACH ROW
WHEN(NEW.ID >0)
DECLARE
    sal_diff number;
BEGIN
    sal_diff :=:NEW.salary  -:OLD.salary;
    dbms_output.put_line('Old salary: '||:OLD.salary);
    dbms_output.put_line('New salary: '||:NEW.salary);
    dbms_output.put_line('Salary difference: '|| sal_diff);
END;
```

When the above code is executed at SQL prompt, it produces the following result:

Trigger created.

Here following two points are important and should be noted carefully:

- OLD and NEW references are not available for table level triggers, rather you can use them for record level triggers.
- If you want to query the table in the same trigger, then you should use the AFTER keyword, because triggers can query the table or change it again only after the initial changes are applied and the table is back in a consistent state.
- Above trigger has been written in such a way that it will fire before any DELETE or INSERT or UPDATE operation on the table, but you can write your trigger on a single or multiple operations, for example BEFORE DELETE, which will fire whenever a record will be deleted using DELETE operation on the table.

4.21.4.5 Triggering a Trigger

Let us perform some DML operations on the CUSTOMERS table. Here is one INSERT statement, which will create a new record in the table:

```
INSERTINTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)
VALUES(7,'Kriti',22,'HP',7500.00);
```

When a record is created in CUSTOMERS table, above create trigger **display_salary_changes** will be fired and it will display the following result:

```
Old salary:
New salary: 7500
Salary difference:
```

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Because this is a new record, old salary is not available and above result is displayed as null. Now, let us perform one more DML operation on the CUSTOMERS table. Here is one UPDATE statement, which will update an existing record in the table:

UPDATE customers SET salary = salary +500
WHERE id =2;

When a record is updated in CUSTOMERS table, above create trigger **display_salary_changes** will be fired and it will display the following result:

Old salary: 1500
New salary: 2000
Salary difference: 500

4.21.4.6 Disabling Triggers

Disabling trigger statement helps in disabling the created triggers

Syntax: *Alter trigger <trigger_name> disable;*

Example: Alter trigger display_salary_changes disable;

Specific triggers on a table can be disabled as follows

Syntax: ***Alter*** *table <table_name> disable <trigger_name>;*

Example: Alter table cutomers disable display_salary_changes;

All triggers on a table can be disabled on a table as follows.

Syntax: ***Alter*** *table <table_name> disable all triggers;*

Example: **Alter** table customers disables all triggers;

4.21.4.7 Enabling Trigger

Enabling trigger statement helps in enabling the created triggers

Syntax: ***Alter*** *table <table_name> enable trigger_name;*

Example: **Alter** table customers enable display_salary_changes;

To enable all triggers

Syntax :*Alter table <table_name> enable all triggers;*

4.21.4.8 Dropping triggers

This statement helps to drop the created trigger. Once this statement is executed the trigger created will be no longer be present.

Syntax: ***Drop*** *trigger <trigger_name>;*

Example: **Drop** trigger display_salary_changes;

4.21.5 Indexes

Indexes are special lookup tables that the database search engine can use to speed up data retrieval. An index is a pointer to data in a table basically used for performance tuning and fast retrieval. An index in a database is very similar to an index in the back of a book.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

For example, if you want to refer all pages in a book that discuss a certain topic, you first refer to the index which lists all topics alphabetically and are then referred to one or more specific page numbers.

An index helps speed up SELECT queries and WHERE clauses, but it slows down data input, with UPDATE and INSERT statements. Indexes can be created or dropped with no effect on the data.

Creating an index involves the CREATE INDEX statement, which allows you to name the index, to specify the table and which column or columns to index, and to indicate whether the index is in ascending or descending order.

Indexes can also be unique, similar to the UNIQUE constraint, in that the index prevents duplicate entries in the column or combination of columns on which there's an index.

4.21.5.1 The CREATE INDEX Command

The basic syntax of CREATE INDEX is as follows:

CREATE INDEX index_name ON table_name;

Single-Column Indexes: A single-column index is one that is created based on only one table column. The basic syntax is as follows:

CREATE INDEX index_name

ON table_name (column_name);

Unique Indexes: Unique indexes are used not only for performance, but also for data integrity. A unique index does not allow any duplicate values to be inserted into the table. The basic syntax is as follows:

CREATE INDEX index_name

on table_name (column_name);

Composite Indexes: A composite index is an index on two or more columns of a table. The basic syntax is as follows:

CREATE INDEX index_name

on table_name (column1, column2);

Whether to create a single-column index or a composite index, take into consideration the column(s) that you may use very frequently in a query's WHERE clause as filter conditions. If only one column is frequently used, then a single-column index can be used. Whereas if two or more columns are frequently used in the WHERE clause as filters, the composite index would be the best choice.

Implicit Indexes: Implicit indexes are indexes that are automatically created by the database server when an object is created. Indexes are automatically created for primary key constraints and unique constraints.

4.21.5.2 The DROP INDEX Command

An index can be dropped using SQL **DROP** command. Care should be taken when dropping an index because performance may be slowed or improved.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

The basic syntax is as follows:

DROP INDEX index_name;

4.21.5.3 When should indexes be avoided?

Although indexes are intended to enhance a database's performance, there are times when they should be avoided. The following guidelines indicate when the use of an index should be reconsidered:

- Indexes should not be used on small tables.
- Tables that have frequent, large batch update or insert operations.
- Indexes should not be used on columns that contain a high number of NULL values.
- Columns that are frequently manipulated should not be indexed.

4.22 DCL (DATA CONTROL LANGUAGE)

DCL commands are used to enforce database security in a multiple user database environment. Two types of DCL commands are

1. GRANT
2. REVOKE.

Only Database Administrator's or owner's of the database object can provide/remove privileges on a database object.

4.22.1 GRANT Command

SQL GRANT is a command used to provide access or privileges on the database objects to the users.

The Syntax for the GRANT command is:

```
GRANT privilege_name
ON object_name
TO {user_name |PUBLIC |role_name}
[WITH GRANT OPTION];
```

- ***privilege_name*** is the access right or privilege granted to the user. Some of the access rights are ALL, EXECUTE, and SELECT.
- ***object_name*** is the name of an database object like TABLE, VIEW, STORED PROC and SEQUENCE.
- ***user_name*** is the name of the user to whom an access right is being granted.
- ***PUBLIC*** is used to grant access rights to all users.
- ***ROLES*** are a set of privileges grouped together.
- ***WITH GRANT OPTION*** - allows a user to grant access rights to other users.

For Example:

GRANT SELECT ON employee TO user1;

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

This command grants a SELECT permission on employee table to user1.You should be cautious in using the WITH GRANT option because, for example, if you GRANT SELECT privilege on employee table to user1 using the WITH GRANT option, then user1 can GRANT SELECT privilege on employee table to another user, such as user2 etc. Later, if you REVOKE the SELECT privilege on employee from user1, still user2 will have SELECT privilege on employee table.

4.22.2 REVOKE Command

The REVOKE command removes user access rights or privileges to the database objects.

The Syntax for the REVOKE command is:

REVOKE privilege_name
ON object_name
FROM {user_name |PUBLIC |role_name}

For Example:

REVOKE SELECT ON employee FROM user1;

This command will REVOKE a SELECT privilege on employee table from user1.When you REVOKE SELECT privilege on a table from a user, the user will not be able to SELECT data from that table anymore. However, if the user has received SELECT privileges on that table from more than one users, he/she can SELECT from that table until everyone who granted the permission revokes it. You cannot REVOKE privileges if they were not initially granted by you.

4.22.3 Privileges and Roles

Privileges: Privileges define the access rights provided to a user on a database object. There are two types of privileges.

1. **System privileges:** This allows the user to CREATE, ALTER, or DROP database objects.
2. **Object privileges:** This allows the user to EXECUTE, SELECT, INSERT, UPDATE, or DELETE data from database objects to which the privileges apply.

Few CREATE system privileges are listed below:

System Privileges	*Description*
CREATE object	Allows users to create the specified object in their own schema.
CREATE ANY object	Allows users to create the specified object in any schema.

The above rules also apply for ALTER and DROP system privileges.

Few of the object privileges are listed below:

Object Privileges	*Description*
INSERT	Allows users to insert rows into a table.
SELECT	Allows users to select data from a database object.
UPDATE	Allows user to update data in a table.
EXECUTE	Allows user to execute a stored procedure or a function.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Roles: Roles are a collection of privileges or access rights. When there are many users in a database it becomes difficult to grant or revoke privileges to users. Therefore, if you define roles, you can grant or revoke privileges to users, thereby automatically granting or revoking privileges. You can either create Roles or use the system roles pre-defined by oracle.

Some of the privileges granted to the system roles are as given below:

System Role	*Privileges Granted to the Role*
CONNECT	CREATE TABLE, CREATE VIEW, CREATE SYNONYM, CREATE SEQUENCE, CREATE SESSION etc.
RESOURCE	CREATE PROCEDURE, CREATE SEQUENCE, CREATE TABLE, CREATE TRIGGER etc. The primary usage of the RESOURCE role is to restrict access to database objects.
DBA	ALL SYSTEM PRIVILEGES

4.22.4 Creating Roles

The Syntax to create a role is:

CREATE ROLE role_name
[IDENTIFIED BY password];

For Example: To create a role called "testing" with password as "pwd", the code will be as follows:

CREATE ROLE testing
[IDENTIFIED BY pwd];

It's easier to GRANT or REVOKE privileges to the users through a role rather than assigning a privilege directly to every user. If a role is identified by a password, then, when you GRANT or REVOKE privileges to the role, you definitely have to identify it with the password.

We can GRANT or REVOKE privilege to a role as below.

For example: To grant CREATE TABLE privilege to a user by creating a testing role:

First, create a testing Role

CREATE ROLE testing

Second, grant a CREATE TABLE privilege to the ROLE testing. You can add more privileges to the ROLE.

GRANT CREATE TABLE TO testing;

Third, grant the role to a user.

GRANT testing TO user1;

To revoke a CREATE TABLE privilege from testing ROLE, you can write:

REVOKE CREATE TABLE FROM testing;

The Syntax to drop a role from the database is as below:

DROP ROLE role_name;

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

For example: To drop a role called testing, you can write:

```
DROP ROLE testing;
```

4.23 TCL (TRANSACTION CONTROL LANGUAGE)

TCL statements are used to manage the changes made by DML statements. It allows statements to be grouped together into logical transactions.

- COMMIT: save work done
- SAVEPOINT: identify a point in a transaction to which you can later roll back
- ROLLBACK: restore database to original since the last COMMIT
- SET TRANSACTION: Change transaction options like isolation level and what rollback segment to use

Transactional control commands are only used with the DML commands INSERT, UPDATE and DELETE. They cannot be used while creating tables or dropping them because these operations are automatically committed in the database.

4.23.1 The COMMIT Command

The COMMIT command is the transactional command used to save changes invoked by a transaction to the database. The COMMIT command saves all transactions to the database since the last COMMIT or ROLLBACK command.

The **syntax** for COMMIT command is as follows:

```
COMMIT;
```

Example: Consider the CUSTOMERS table having the following records:

Id	*Name*	*Age*	*Address*	*Salary*
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Following is the example which would delete records from the table having age = 25 and then COMMIT the changes in the database.

```
SQL> DELETE FROM CUSTOMERS WHERE AGE =25;
SQL> COMMIT;
```

As a result, two rows from the table would be deleted and SELECT statement would produce the following result:

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Id	Name	Age	Address	Salary
1	Ramesh	32	Ahmedabad	2000.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

4.23.2 The ROLLBACK Command:

The ROLLBACK command is the transactional command used to undo transactions that have not already been saved to the database. The ROLLBACK command can only be used to undo transactions since the last COMMIT or ROLLBACK command was issued.

The **syntax** for ROLLBACK command is as follows:

ROLLBACK;

Example: Consider the CUSTOMERS table having the following records:

Id	Name	Age	Address	Salary
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Following is the example, which would delete records from the table having age = 25 and then ROLLBACK the changes in the database.

SQL> DELETE FROM CUSTOMERS WHERE AGE =25;
SQL> ROLLBACK;

As a result, delete operation would not impact the table and SELECT statement would produce the following result:

Id	Name	Age	Address	Salary
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

4.23.3 The SAVEPOINT Command

A SAVEPOINT is a point in a transaction when you can roll the transaction back to a certain point without rolling back the entire transaction.

The **syntax** for SAVEPOINT command is as follows:

SAVEPOINT SAVEPOINT_NAME;

This command serves only in the creation of a SAVEPOINT among transactional statements. The ROLLBACK command is used to undo a group of transactions.

The **syntax** for rolling back to a SAVEPOINT is as follows:

ROLLBACK TO SAVEPOINT_NAME;

Following is an example where you plan to delete the three different records from the CUSTOMERS table. You want to create a SAVEPOINT before each delete, so that you can ROLLBACK to any SAVEPOINT at any time to return the appropriate data to its original state:

Example: Consider the CUSTOMERS table having the following records:

Id	*Name*	*Age*	*Address*	*Salary*
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

Now, here is the series of operations:

SQL> SAVEPOINT SP1;
Savepoint created.

SQL> DELETE FROM CUSTOMERS WHERE ID=1;
1 row deleted.
SQL> SAVEPOINT SP2;
Savepoint created.
SQL> DELETE FROM CUSTOMERS WHERE ID=2;
1 row deleted.
SQL> SAVEPOINT SP3;
Savepoint created.

SQL> DELETE FROM CUSTOMERS WHERE ID=3;
1 row deleted.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Now that the three deletions have taken place, say you have changed your mind and decided to ROLLBACK to the SAVEPOINT that you identified as SP2. Because SP2 was created after the first deletion, the last two deletions are undone:

SQL> ROLLBACK TO SP2;

Rollback complete.

Notice that only the first deletion took place since you rolled back to SP2:

SQL> SELECT * FROM CUSTOMERS;

Id	*Name*	*Age*	*Address*	*Salary*
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	Kota	2000.00
4	Chaitali	25	Mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	Muffy	24	Indore	10000.00

6 rows selected.

4.23.4 The RELEASE SAVEPOINT Command

The RELEASE SAVEPOINT command is used to remove a SAVEPOINT that you have created.

The **syntax** for RELEASE SAVEPOINT is as follows:

RELEASE SAVEPOINT SAVEPOINT_NAME;

Once a SAVEPOINT has been released, you can no longer use the ROLLBACK command to undo transactions performed since the SAVEPOINT.

4.23.5 The SET TRANSACTION Command

The SET TRANSACTION command can be used to initiate a database transaction. This command is used to specify characteristics for the transaction that follows.

For example, you can specify a transaction to be read only, or read write.

The **syntax** for SET TRANSACTION is as follows:

SET TRANSACTION [READ WRITE | READ ONLY];

REVIEW QUESTIONS

1. Define SQL.
2. Give the basic structure of SQL expression.
3. List out the different types of SQL statements.
4. Define the following and list out the comments under each of the following.
   (*i*) DDL
   (*ii*) DML

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

(*iii*) DCL
(*iv*) TCL

5. List out the advantages of using SQL statements.
6. Define database objects.
7. What is need of HAVING clause in SQL?
8. Define triggers.
9. What is the use of sequence in SQL?
10. List out the different database objects and define each of them.

Queries

1. Create a table with following table and name it as ACCOUNTS.

Name	*Type*
acc_no	varchar (5)
branch_name	varchar (12)
balance	number(7)

A. Write a query to describe the structure of the ACCOUNTS table.
B. Write a query to determine the contents of the ACCOUNTS table.
C. Write a query to display the branch_name.
D. Find branch_name having balance>30,000.
E. Change the data type of acc_no to number(5).
F. Include a new column branch_id to the ACCOUNTS table.
G. Change the size of balance field from 7 to 10.
H. Truncate the table.
I. Drop the created table.

2. Create the following table and insert the given values into the table:

Customer Table:

cid	*Cust_name*	*Cust_street*	*Cust_city*
1	Jones	Main	Harrison
2	Smith	North	Rye
3	Hays	Main	Princeton
4	Curry	North	Pittsfield
5	Lindsay	Park	Palo Alto

For the questions below consider the employees table used in chapter 4

3. Display the last name concatenated with job_id and name the column Employee and Title.
4. Display the last_name and salary of employees in employees table who earn more than 10,000.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

5. Display the last_name and department number for employee having department_id=100.
6. Display the names of all the employees whose salary lies between 15,000 and 25,000.
7. Find the names of employees whose employee_id is either 100, 102 or 104.
8. Display the names and id of all employees whose name starts with letter 'A'.
9. Display all employees last name in which the third letter of the name is "a.".
10. Display the last name of all employees who have both a and s in their last name.
11. Create a view for employees table and name it as Employees1.
12. Copy the values present in the employees table to the view Employees1 using subquery concept.
13. Create a report to display all details of employees who earn More than $12,000.
14. Display the last_name and salary for any employee whose salary is not in $5000to $12000.
15. Find the Average, minimum and maximum salary of the employees in the employee table.
16. Fine the average salary of the employees in the employee table group by department_id.
17. Fine the minimum salary of the employees in the employee table group by department_id and check that the minimum salary does not fall below 1000.
18. Consider the employees table and departments table from our 4th chapter and perform all the set operations.
19. Create the following Library and summary tables.

    Library table:

Name	*Type*
BNO	NUMBER(1)
BNAME	VARCHAR2(20)
NCOPY	NUMBER(2)

    Summary Table:

Name	*Type*
BNO	NUMBER(1)
NCOPY	NUMBER(2)

    Inert values in to the library table and write a trigger such that BNO and NCOPY are automatically updated in the summary table.
20. Create sequenceseq1 increment by 1 start with 100 maxvalue 105 and invalue 100.

❑❑❑

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.

Copyright © 2016. Alpha Science International. All rights reserved.

Vidhya, V., et al. Database Management Systems, Alpha Science International, 2016. ProQuest Ebook Central,
http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=5248352.
Created from univ-people-ebooks on 2026-05-02 05:46:38.